



CHRONIQO SYSTEMDESIGN & ARKITEKTUR

SW7BAC-01 BACHELORPROJEKT

BILAG 5

Elev:

Mads Dittmann Villadsen

202209869 / AU715368

Vejleder:

Tommy Bjerre Nielsen

tbn@ece.au.dk

MAJ 2026

Indholdsfortegnelse

Figurliste	iii
Tabelliste	iv
1. Indledning	1
1.1. Formål med systemdesign og arkitektur	2
1.2. Placering i udviklingsprocessen	2
2. Systemarkitektur (C4-modellen)	3
2.1. System Context (level 1)	4
2.2. Container Diagram (level 2)	6
2.2.1. Arkitektoniske overvejelser på containerniveau	8
2.3. Component Diagram (level 3)	9
2.3.1. Arkitektoniske overvejelser på komponentniveau	11
2.4. Deployment arkitektur	12
3. Databasesdesign	14
3.1. Entity-Relationship (ER) Model	15
3.1.1. Sub-figur A: Auth, identitet og sikkerhed	17
3.1.2. Sub-figur B: Social graf	19
3.1.3. Sub-figur C: Fællesskaber og adgangsstyring	21
3.1.4. Sub-figur D: Indhold: Post og Comment	23
3.1.5. Sub-figur E: Moderation og notifikationer	25
3.1.6. Sub-figur F: Chat, minispil og AI-assistent	27
3.2. Data Dictionary (tabelspecifikation)	30
4. Applikationsmodel (softwaredesign)	43
4.1. Designmønstre og arkitektoniske valg	44
4.2. Design klassesdiagram (UML)	45
4.2.1. Autentificering og sikkerhed (Auth Domain)	45
4.2.2. Kernedomænet (dagsform og socialt)	47
4.2.3. Kommunikationsdomænet (chat, minispil og AI)	48
5. Systemsekvensdiagrammer (dynamisk adfærd)	50
5.1. SD: Brugeroprettelse og onboarding (US1.1)	51

5.2. SD: Autentificeringsflow med OAuth 2.0 PKCE (US1.2/1.3)	53
5.3. SD: Registrering af dagsform (US2.1).....	55
5.4. SD: Opret opslag anonymt (US3.10)	57
5.5. SD: Middleware-afvisning (rate limit/CSRF) (NFR)	59
5.6. SD: Realtidsreaktioner og Optimistic UI (US3.3)	61
5.7. SD: Oprettelse af fællesskab (US3.7)	63
5.8. SD: Anmodning om medlemskab i privat fællesskab (US3.14)	65
5.9. SD: Gem og hent opslagskladde (afledt af US3.8)	67
5.10. SD: Personlig skjulning af fællesskab (afledt af US3.14/US2.13)	69
5.11. SD: AI-støttebot med privacy-filtrering (US3.19)	71
6. API-design (grænseflader)	73
6.1. REST API kontrakter	74
7. Referencer	78
8. Bilag	80

Figurliste

Figur 1: C4 System Context Diagram (Level 1) for Chroniqo	5
Figur 2: C4 Container Diagram (Level 2) for Chroniqo	7
Figur 3: C4 Component Diagram (Level 3) for Next.js applikationen for Chroniqo	10
Figur 4a: Komplet Entity-Relationship (ER) diagram for Chroniqos database.....	16
Figur 4b (Sub-figur A): ER-diagram for autentificerings- og identitetsdomænet	18
Figur 4c (Sub-figur B): ER-diagram for den sociale graf.....	20
Figur 4d (Sub-figur C): ER-diagram for fællesskabsdomænet.....	22
Figur 4e (Sub-figur D): ER-diagram for indholdsdomænet.....	24
Figur 4f (Sub-figur E): ER-diagram for moderations- og notifikationsdomænet.....	26
Figur 4g (Sub-figur F): ER-diagram for kommunikationsdomænet.....	29
Figur 5: Design-klassediagram for autentificeringsmodulet.....	46
Figur 6: Design-klassediagram for systemets kernerdomæne.....	47
Figur 7: Design-klassediagram for kommunikationsdomænet	49
Figur 8: Sekvensdiagram for brugeroprettelse og onboarding (US1.1).....	52
Figur 9: Sekvensdiagram for autentificeringsflow med OAuth 2.0 PKCE (US1.2/1.3).....	54
Figur 10: Sekvensdiagram for registrering af dagsform (US2.1)	56
Figur 11: Sekvensdiagram for opret opslag anonymt (US3.10)	58
Figur 12: Sekvensdiagram for Edge Middleware-afvisning, rate limit (NFR)	60
Figur 13: Sekvensdiagram for realtidsreaktioner og Optimistic UI (US3.3).....	62
Figur 14: Sekvensdiagram for oprettelse af fællesskab (US3.7)	64
Figur 15: Sekvensdiagram for anmodning om medlemskab i et privat fællesskab (US3.14)	66
Figur 16: Sekvensdiagram for håndtering af opslagskladder.....	68
Figur 17: Sekvensdiagram for personlig skjulning af fællesskab	70
Figur 18: Sekvensdiagram for AI-støttebotten Niqo med PII privacy-filtrering (US3.19).....	72

Tabelliste

Tabel 1: Oversigt over deployment-arkitektur og serverless-infrastruktur for Chroniqo	13
Tabel 2: Data Dictionary - specifikation af alle 40 tabeller	31
Tabel 3: Oversigt over systemets flerlagshierarki	44
Tabel 4: Specifikation af centrale REST API-endpoints	75

1. Indledning

Dette dokument udgør systemdesignet og arkitekturbeskrivelsen for Chroniqo. Hvor den forudgående kravspecifikation og domæneanalyse har fokuseret på at afdække, hvad systemet skal kunne, fokuserer dette dokument på, hvordan systemet skal konstrueres rent teknisk.

Dokumentet omsætter de identificerede brugerbehov, forretningsregler (de logiske betingelser og restriktioner, der definerer domænet - f.eks. at et Opslag altid kræver en ejer) og kvalitetskrav til en konkret, teknisk byggeplan. Gennem anvendelsen af anerkendte modelleringsstandarder defineres systemets strukturelle opbygning og dynamiske adfærd, hvilket danner det direkte fundament for den efterfølgende implementering.

1.1. Formål med systemdesign og arkitektur

Formålet med dette dokument er at etablere en robust, skalerbar og sikker arkitektur for Chroniqo. Som enkeltmandsudvikler med et begrænset tidsestimat er det afgørende at træffe velovervejede arkitektoniske beslutninger tidligt i processen, da omkostningerne ved at rette fundamentale designfejl stiger eksponentielt, jo længere man er i udviklingsforløbet.

Dokumentet tjener tre primære formål:

- **Strukturelt overblik:** At visualisere systemets opbygning og integrationer via C4-modellen, fra det overordnede systemkontekst-niveau (level 1) ned til de enkelte deployerbare containere og komponenter (level 2 og 3).
- **Dataintegritet:** At transformere den konceptuelle domæne-model til et fysisk, relationelt databasedesign (ER-model), der håndhæver domænets forretningsregler og understøtter systemets krav til datakonsistens.
- **Adfærdsverifikation:** At validere systemets dynamiske flow og grænseflade-kommunikation (API-design) gennem systemsekvensdiagrammer af udvalgte, arkitektonisk centrale Use Cases og sikkerhedsmekanismer.

1.2. Placering i udviklingsprocessen

I henhold til den anvendte ASE-udviklingsmodel (1), placerer dette dokument sig i designfasen - i brobygningen mellem analyse og implementering.

Dokumentet trækker direkte tråde tilbage til projektets tidligere faser. Arkitekturvalgene og infrastrukturen er dikteret af Teknologianalysen, risikohåndteringen (såsom implementering af rate limiting og sikkerhedsmiddleware) er afledt direkte af Risikoanalysen, og den logiske opbygning af applikationsmodellen bygger 1:1 på koncepterne etableret i Domæneanalysen (2-4).

I overensstemmelse med ASE-modellens iterative natur er designet ikke blevet behandlet som en statisk facitliste, men som en dynamisk baseline. Selvom designet løbende er blevet tilpasset i takt med, at tekniske udfordringer og edge cases opstod under implementeringen, afspejler dette dokument det endelige design, der svarer til det færdige system.

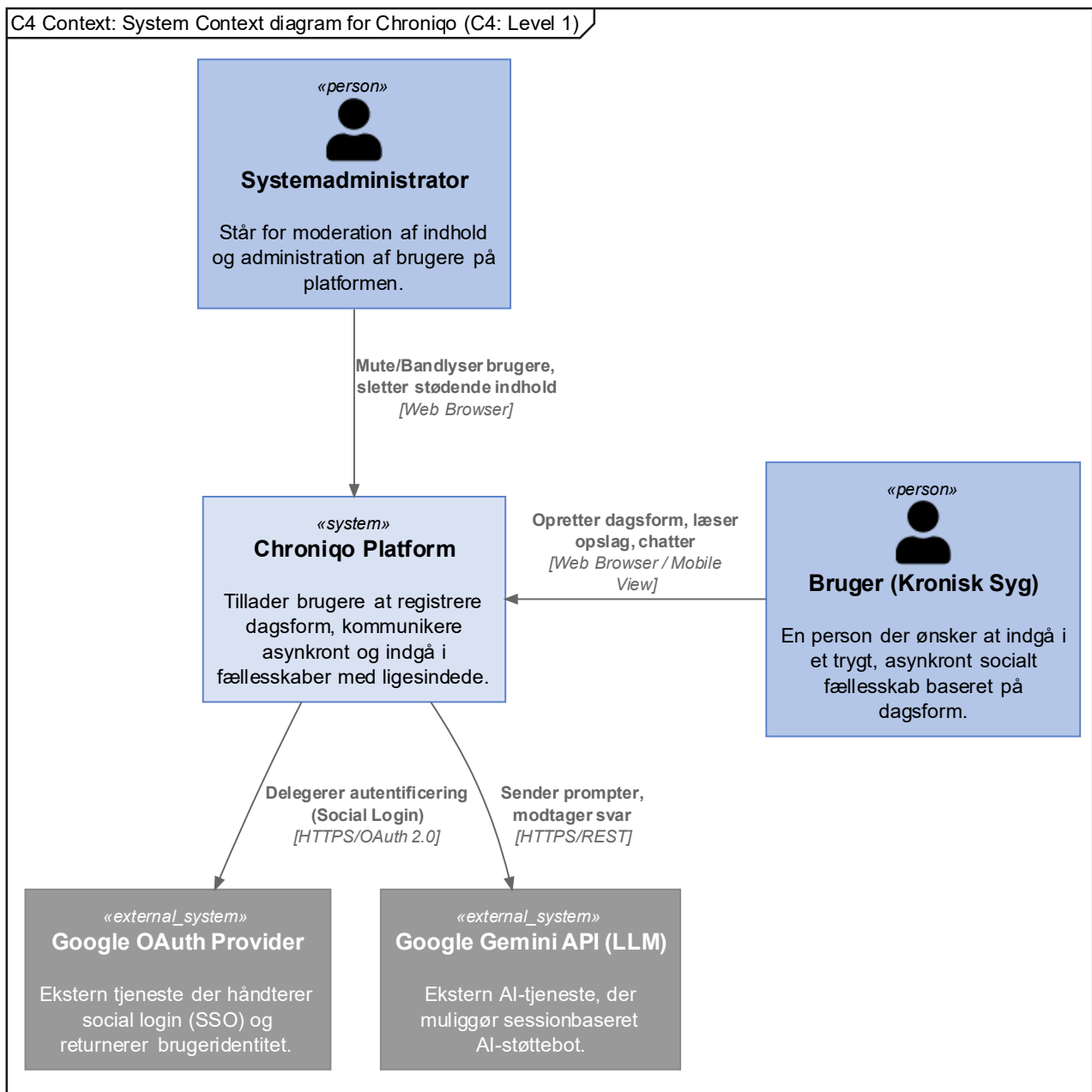
2. Systemarkitektur (C4-modellen)

For at skabe et entydigt og struktureret overblik over Chroniqos arkitektur, anvendes C4-modellen (Context, Containers, Components, og Code) (5). Modellen tilbyder en standardiseret, hierarkisk tilgang til at visualisere softwarearkitektur gennem forskellige abstraktionsniveauer, hvor hvert niveau dykker længere ned i systemets tekniske detaljer.

Som afgrænset i indledningen, stoppes modelleringen bevidst ved Level 3 (Component Diagram). Det detaljerede kodeniveau (Level 4) er udeladt, da denne funktionalitet overlapper direkte med dokumentets senere afsnit vedrørende Applikationsmodellen (Design Class Diagram), som tilbyder et mere præcist UML-sprog til modellering af specifikke attributter og metoder.

2.1. System Context (level 1)

System Context-diagrammet repræsenterer det højeste abstraktionsniveau i C4-modellen. Formålet er at betragte Chroniqo som en samlet black box og udelukkende fokusere på systemets grænseflader til de primære aktører og eksterne systemer. Dette niveau bygger direkte videre på aktøranalysen fra kravspecifikationen, som det fremgår af Figur 1 på efterfølgende side:

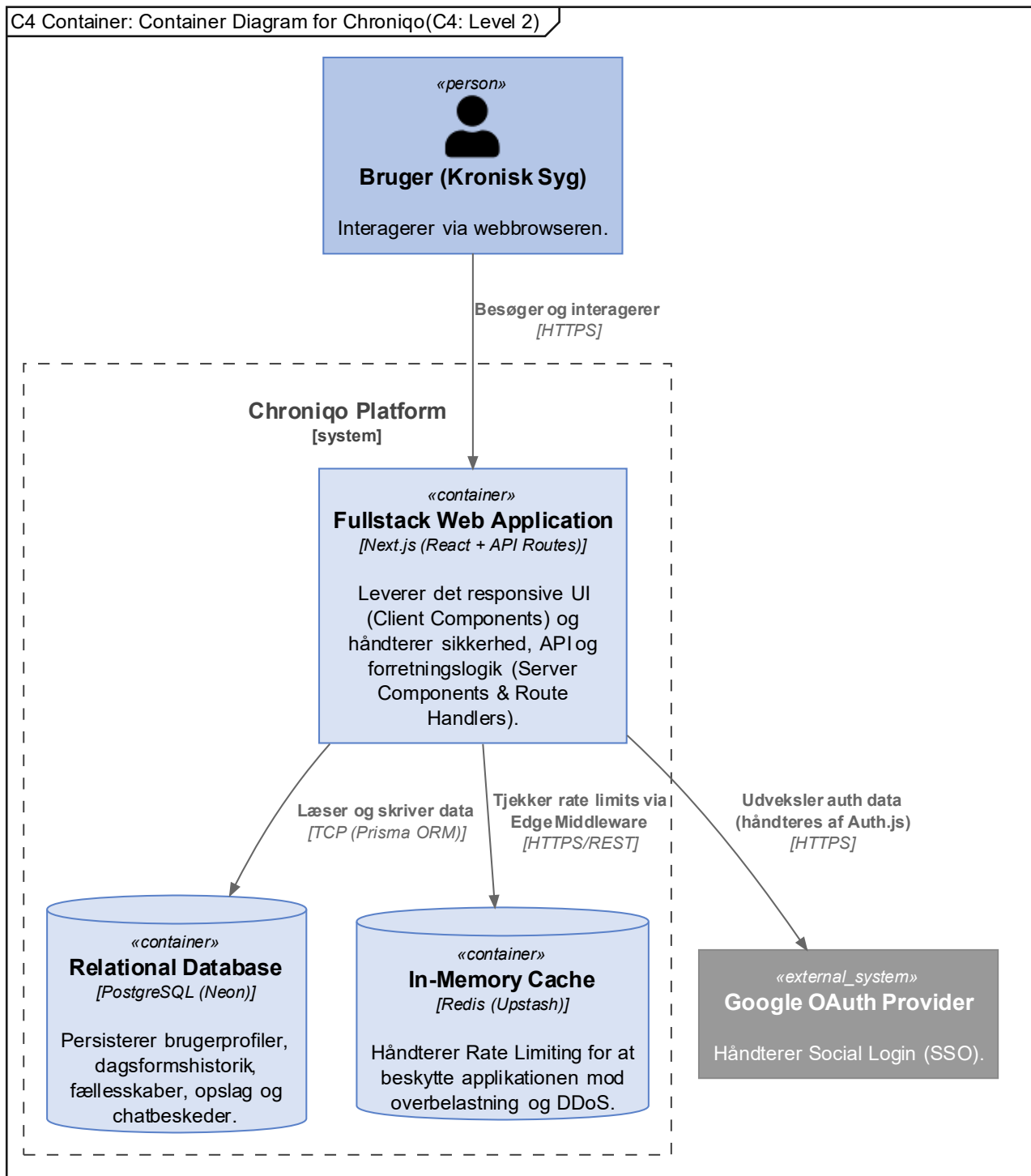


Figur 1: C4 System Context-diagram (Level 1) for Chroniqo. Diagrammet illustrerer, hvordan brugeren og systemadministratoren interagerer med den samlede platform, samt hvordan systemet integrerer med eksterne tjenester som Google OAuth Provider til autentificering af brugere via OAuth 2.0.

2.2. Container Diagram (level 2)

Container-diagrammet åbner platformens black box og viser de overordnede, uafhængigt eksekverbare enheder, som systemet består af. På dette niveau visualiseres de strategiske valg fra teknologi-analysen, hvor projektet er bygget som en samlet Next.js Fullstack-applikation (2).

Diagrammet illustrerer desuden den samlede data-infrastruktur, herunder den eksterne afhængighed til Google OAuth via Auth.js, hvilket er visualiseret i Figur 2 på efterfølgende side.



Figur 2: C4 Container-diagram (Level 2) for Chroniqo. Viser den arkitektoniske samling af platformens kernekomponenter i én Next.js webapplikation, understøttet af PostgreSQL via Prisma og Upstash Redis til rate limiting.

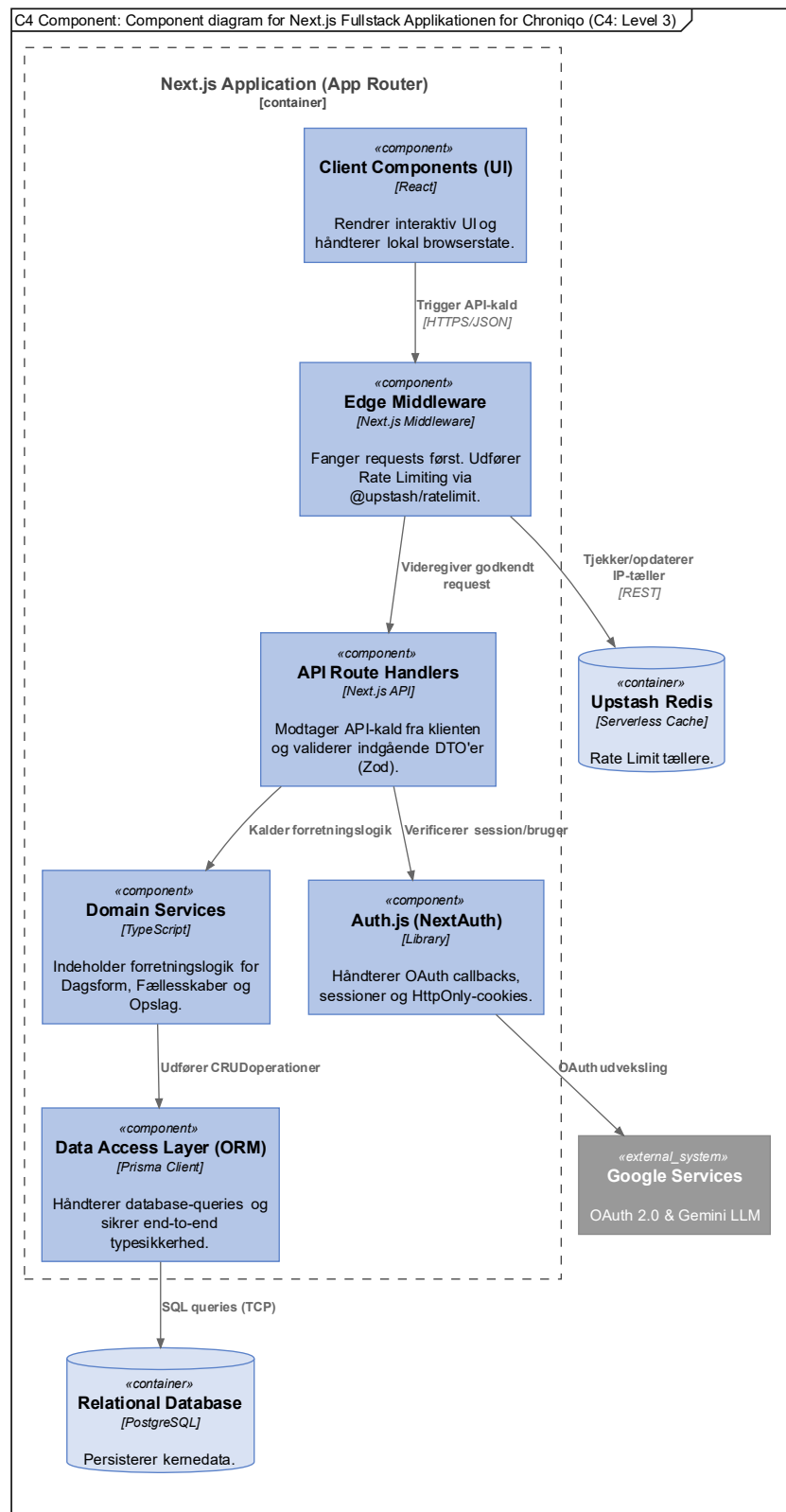
2.2.1. Arkitektoniske overvejelser på containerniveau

Det valgte design samler klientlogik og backend-logik i ét samlet Next.js framework (Monolit) (6). Dette designvalg er truffet for at minimere den kognitive belastning og den administrative kompleksitet, der er forbundet med at vedligeholde to separate kodebaser. Ved at reducere denne type arkitektoniske friktion, imødegås den primære risikofaktor i projektet (jf. R1: Helbred/Fatigue (3)) direkte gennem en forenklet arbejdsgang.

Sikkerhedsmæssigt udnyttes Next.js' arkitektur til at holde følsomme operationer på serveren. Autentificering og session-håndtering er udliciteret til Auth.js (NextAuth) (7), som automatisk sørger for sikker udveksling af OAuth-tokens med Google og etablering af sikre `HttpOnly`-cookies (8) til klienten. For at beskytte systemet mod brute-force-angreb (jf. R13: Sikkerhedshuller (3)) benyttes en serverless Redis-løsning via Upstash (9, 10). Rate limiting håndteres derved i Next.js Edge Middleware (11), hvilket sikrer, at ondsindet trafik afvises før det rammer applikationens kerne-logik.

2.3. Component Diagram (level 3)

Component-diagrammet dykker ned i Next.js-applikationen for at belyse dens interne logiske opbygning. Formålet er at illustrere den præcise adskillelse af ansvarsområder (Separation of Concerns), som sikrer, at monolittens forskellige funktioner holdes adskilt og overskuelige, hvilket fremgår af Figur 3 på den efterfølgende side:



Figur 3: C4 Component-diagram (Level 3) for Next.js applikationen for Chroniqo. Diagrammet illustrerer anmodningsvej fra Client Components, gennem Edge Middleware (rate limiting), videre til Route Handlers og Services, og til sidst ned i databasen via Prisma.

2.3.1. Arkitektoniske overvejelser på komponentniveau

Selvom systemet er realiseret som en monolit, er koden struktureret efter et flerlagret arkitekturmønster (Layered Architecture) (12). Her fungerer Next.js Route Handlers (13) udelukkende som indgangsvinkler (Controllers), der håndterer HTTP-anmodninger og DTO-validering via Zod (14, 15). Al forretningslogik er isoleret i et dedikeret Service-lag, mens databasetilgængeligheden håndteres gennem Prisma ORM (16). Dette sikrer en ubrudt typesikkerhed fra databaseskemaet til frontend-lagets React-komponenter, hvilket optimerer systemets testbarhed og vedligeholdelsesvenlighed (Supportability).

2.4. Deployment arkitektur

Deployment-arkitekturen redegør for, hvordan systemets logiske containere eksekveres i et cloud-miljø. For at understøtte projektets målsætning om en skalerbar MVP uden driftsmæssige omkostninger eller vedligeholdelsesbyrde, er der valgt en fuld serverless-strategi som besluttet i Teknologianalysen (2). Denne tilgang muliggør en NoOps-model, hvor fokus flyttes fra infrastrukturstyring til ren applikationsudvikling, som opsummeret i Tabel 1 på næste side.

Tabel 1: Oversigt over deployment-arkitektur og serverless-infrastruktur for Chroniqo. Tabellen specificerer de valgte platforme og deres tekniske roller i arkitekturen.

Deployment arkitektur		
Komponent	Platform	Rolle og funktionalitet
Fullstack Ap- plikation	Vercel (17)	Eksekverer Next.js-applikationen (React UI og API Route Handlers). Vercels/Next.js' Edge Network minimerer latens og håndterer systemets Middleware tæt på slutbrugeren.
Database	Neon.tech (18)	Leverer en PostgreSQL-instans med scale-to-zero funktionalitet. Inkluderer indbygget connection-pooling, som er essentiel for at håndtere kortlivede serverless-forbindelser.
Cache & Rate-Limit	Upstash (10)	Fungerer som en serverless Redis-instans til rate limiting. Integreres direkte med Edge Middleware (11) for proaktivt at beskytte applikationen mod overbelastning.

Denne distribuerede, men managed, deployment-strategi isolerer potentielle fejlkilder (jf. Risiko R14: Problemer relateret til systemets deployment (3)) og sikrer, at systemet kan opdateres automatisk via en CI/CD-pipeline direkte fra GitLab, uden at udvikleren skal opsætte eller vedligeholde traditionelle servermiljøer.

3. Databasedesign

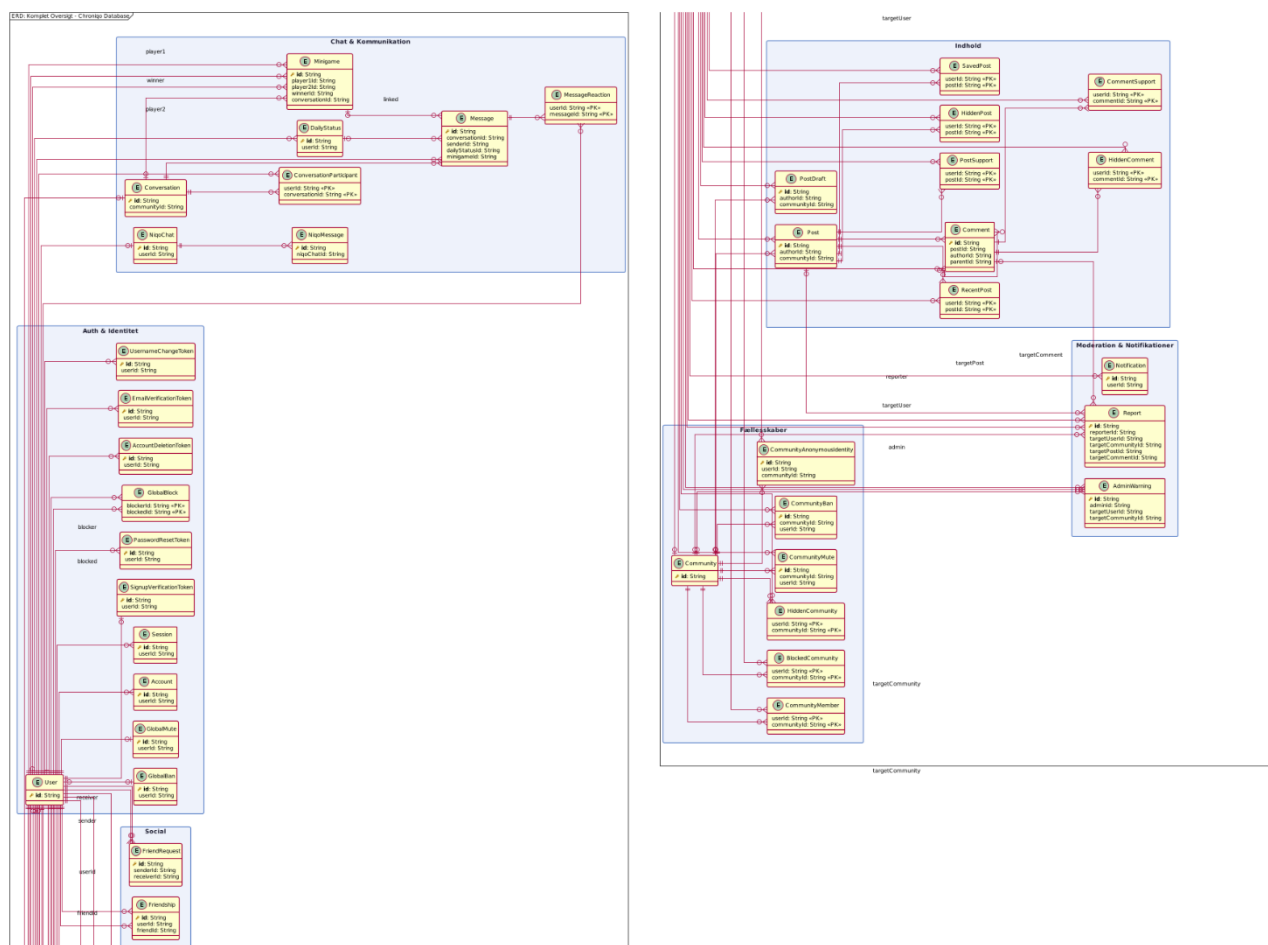
Dette afsnit beskriver oversættelsen af den konceptuelle domænemodel til et fysisk, relationelt databasedesign. Designet er målrettet PostgreSQL (19) og modelleres med udgangspunkt i Prisma ORM's skemastruktur (16). Formålet med at specificere det fysiske design før implementeringen er at sikre, at domænets forretningsregler - herunder dataintegritet og regler for sletning (Cascading) - håndhæves på lavest mulige niveau i arkitekturen.

3.1. Entity-Relationship (ER) Model

ER-modellen for Chroniqo, illustreret i Figur 4a, viser systemets tabeller, primærnøgler (PK), fremmednøgler (FK) og de indbyrdes relationsforhold (kardinaliteter). Databasen er omfattende og består i sin endelige form af 40 modeller, der understøtter alt fra sikkerhed og moderation til asynkron spil og AI-integration.

Arkitekturen integrerer to overordnede domæner: Sikkerhed (Auth.js) og Forretningslogik. For at understøtte Auth.js' indbyggede Prisma Adapter er der implementeret standard-tabellerne `Account` og `Session`, som knytter sig til `User` (7, 20). Forretningslogikken fra domæneanalysen (4) udvides med fysiske koblingstabeller (junction tables) - herunder `CommunityMember`, `ConversationParticipant` og `MessageReaction` - for at håndtere mange-til-mange (N:M) relationer korrekt og uden redundans (21).

Da den samlede model er for omfangsrig til at præsentere læsbart i ét diagram, er den overordnede Figur 4a placeret som en komplet strukturel referenceoversigt, mens modellens detaljer - samtlige felter og constraints - præsenteres i seks logiske domæne-udsnit (sub-figurerne A-F) i de efterfølgende underafsnit.



Figur 4a: Komplet Entity-Relationship (ER) diagram for Chroniqos database. Diagrammet giver en strukturel oversigt over alle 40 modeller og samtlige relationer i systemet. Kolonnedetaljer er udeladt af læsbarhedshensyn og fremgår i stedet af Sub-figurerne A - F i de efterfølgende underafsnit.

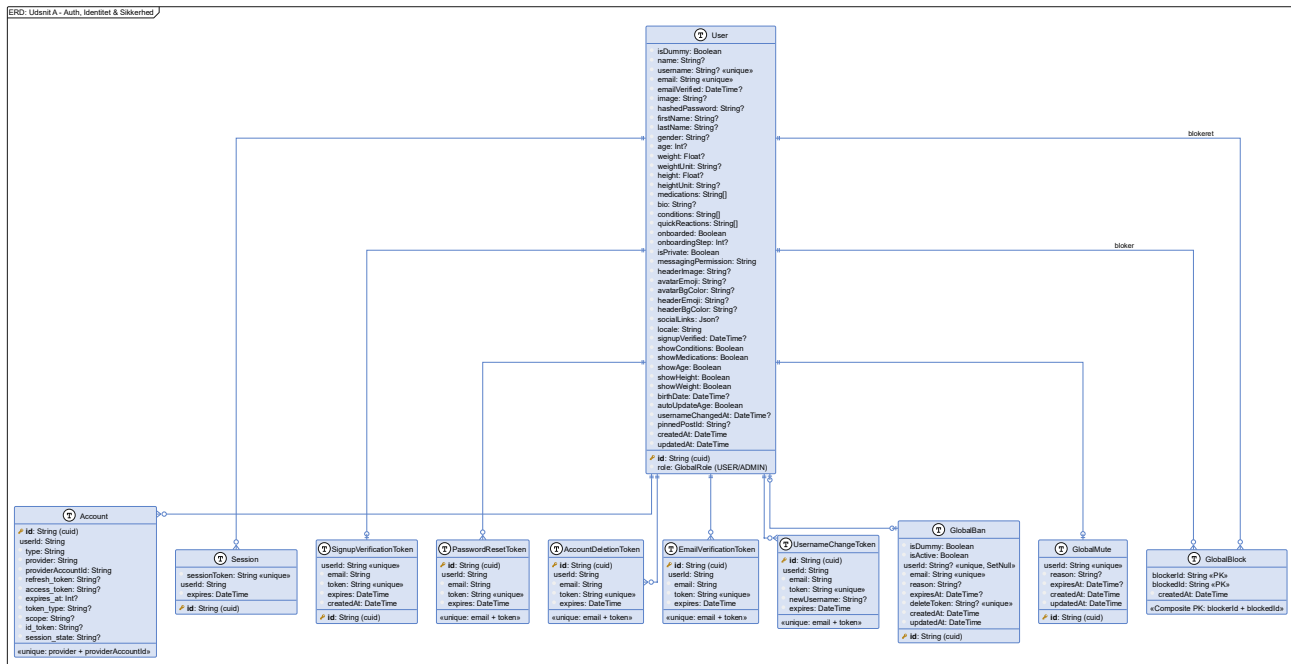
3.1.1. Sub-figur A: Auth, identitet og sikkerhed

Dette udsnit af ER-diagrammet, illustreret i Figur 4b, dækker det samlede autentificerings- og identitetsdomæne. `User`-modellen udgør systemets absolutte centrum og fungerer som det fælles referencpunkt for samtlige øvrige modeller. Tabellen opbevarer alt fra loginoplysninger og rettigheder (via `GlobalRole`-enum) til profildata, personlige indstillinger og synlighedsflags for sundhedsoplysninger.

`Auth.js`-tabellerne `Account` og `Session` er obligatoriske tekniske modeller, der kræves af bibliotekets Prisma Adapter (20). `Account` lagrer forbindelsen til en ekstern OAuth-udbyder (f.eks. Google), mens `Session` administrerer aktive bruger-sessioner. Begge kaskadeslettes (`ON DELETE CASCADE`), hvis den tilhørende `User` fjernes (16).

For at understøtte systemets sikkerhedsflows er der modelleret fem dedikerede token-tabeller, hver med et præcist, afgrænset ansvarsområde: `SignupVerificationToken` til e-mailbekræftelse ved oprettelse, `PasswordResetToken` til adgangskode-nulstilling, `AccountDeletionToken` til bekræftelse af kontosletning, `EmailVerificationToken` til ændring af e-mailadresse og `UsernameChangeToken` til håndtering af brugernavnsskift inklusiv den 30-dages spærreperiode. Alle fem tokens har en `expires`-kolonne, der sikrer automatisk udløb, og er tilknyttet `User` via `ON DELETE CASCADE` (16).

Endelig dækker dette udsnit tre globale moderationsmodeller. `GlobalBan` bandlyser en bruger på platformniveau og er designet til at persistere selv efter kontosletning - `userId`-relationen anvender `onDelete: SetNull`, mens `email`-feltet bevares som det permanente identifikationspunkt, hvilket sikrer, at en bandlysning forbliver aktiv selvom brugerkontoen slettes (16). `GlobalMute` dæmper en brugers publiceringsrettigheder midlertidigt og kaskadeslettes ved kontosletning. `GlobalBlock` er en koblingstabel med sammensat primærnøgle (`blockerId` + `blockedId`), der håndterer bruger-til-bruger-blokeringer bilateralt (16).



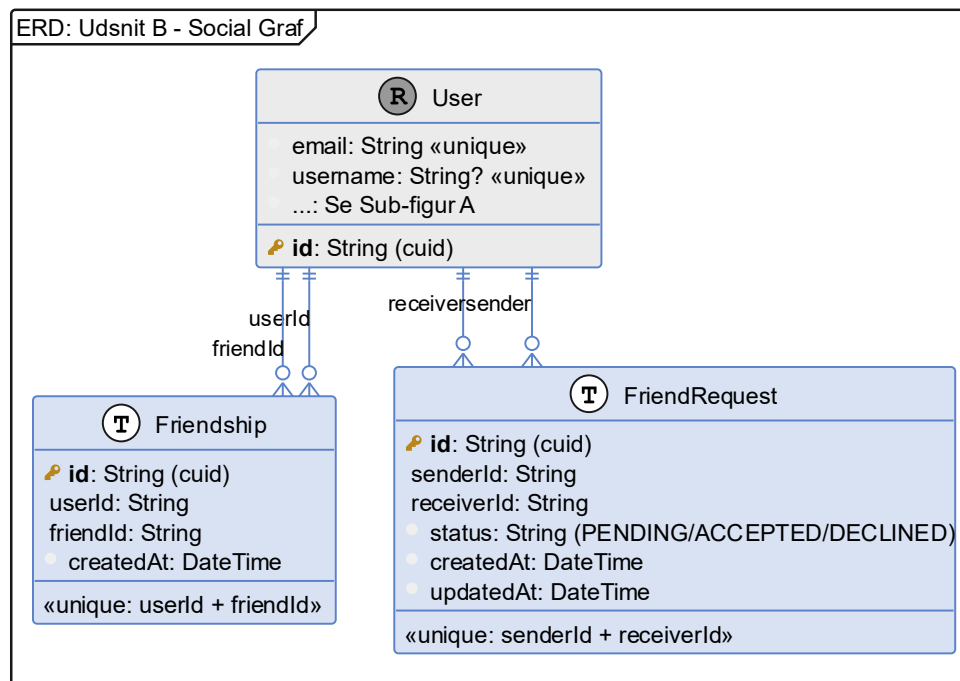
Figur 4b (Sub-figur A): ER-diagram for autentificerings- og identitetsdomænet. Viser User-modellen med alle felter inklusiv profildata og synlighedsflags, Auth.js-modellerne Account og Session, samt alle verifikations- og token-modeller (SignupVerificationToken, PasswordResetToken, AccountDeletionToken, EmailVerificationToken, UsernameChangeToken) samt globale moderationsmodeller (GlobalBan, GlobalMute, GlobalBlock).

3.1.2. Sub-figur B: Social graf

Dette udsnit af ER-diagrammet, illustreret i Figur 4c, beskriver den sociale graf, der forbinder brugerne med hinanden. Den er intentionelt holdt som et selvstændigt domæne for at tydeliggøre adskillelsen af den sociale infrastruktur fra autentificeringen.

`Friendship` repræsenterer en etableret, bidirektionel venskabsrelation. Modellen anvender to separate fremmednøgler til `User` - `userId` og `friendId` - for at modellere relationens retningsbestemmelse korrekt på databaseniveau (21). En Composite Unique Constraint på [`userId`, `friendId`] forhindrer duplikerede relationer (16). Begge fremmednøgler er konfigureret med `ON DELETE CASCADE`, så relationen automatisk ryddes op, hvis en af de tilknyttede konti slettes.

`FriendRequest` modellerer den asynkrone godkendelsesproces, der leder frem til en etableret venskabsrelation. Et `status`-felt med de tre tilstande `PENDING`, `ACCEPTED` og `DECLINED` driver tilstandsmaskinen. En Composite Unique Constraint på [`senderId`, `receiverId`] forhindrer, at en bruger sender adskillige anmodninger til den samme modtager (16). `ON DELETE CASCADE` på begge fremmednøgler sikrer oprydning, hvis afsender eller modtager slettes.



Figur 4c (Sub-figur B): ER-diagram for den sociale graf. Viser Friendship- og FriendRequest-modellerne, der tilsammen understøtter det bidirektionelle venskabssystem med tilhørende anmodningsflow (PENDING til ACCEPTED / DECLINED). Begge modeller er relateret til User via separate fremmednøgler for at modellere den retningsbestemte relation korrekt.

3.1.3. Sub-figur C: Fællesskaber og adgangsstyring

Dette udsnit af ER-diagrammet, illustreret i Figur 4d, beskriver den del af datamodellen, der understøtter fællesskaber og den tilhørende adgangsstyring.

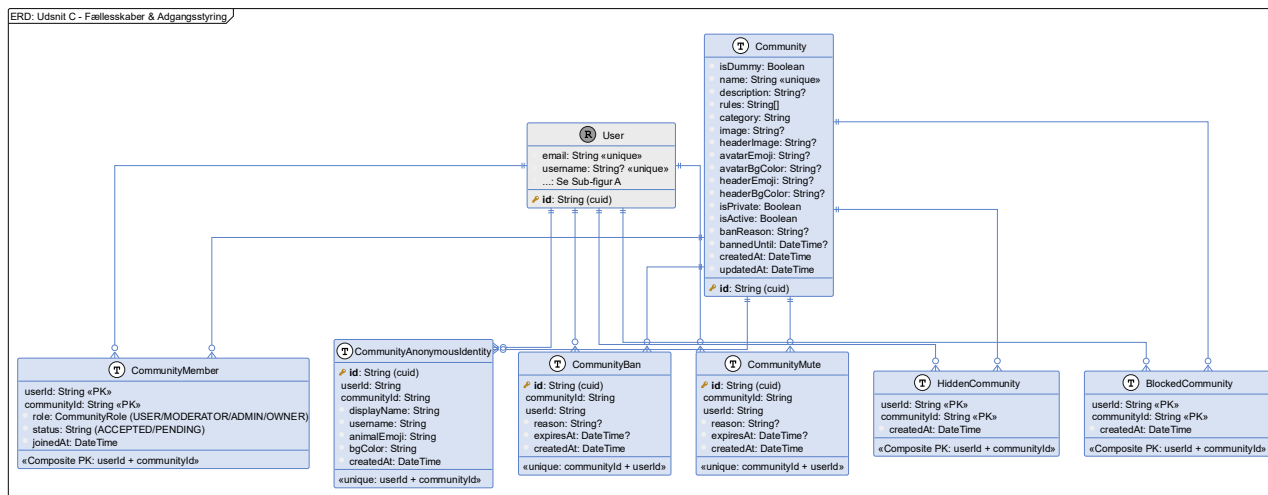
`Community` er den centrale enhed og definerer de logiske fællesskaber, som brugere samles i. Modellen rummer metadata om fællesskabets konfiguration - herunder `isPrivate`, `isActive` og `category` - samt understøttelse af suspension på systemniveau via `banReason` og `bannedUntil`.

`CommunityMember` er den primære koblingstabel (junction table) med sammensat primærnøgle (`userId` + `communityId`), der modellerer N:M-relationen mellem brugere og fællesskaber. Udover tilknytningen indeholder tabellen `role` (via `CommunityRole`-enum: `USER`, `MODERATOR`, `ADMIN`, `OWNER`) og `status` (`ACCEPTED` / `PENDING`), hvilket muliggør en præcis, rollebaseret adgangsstyring.

`CommunityAnonymousIdentity` er en kritisk model for platformens kernefunktionalitet. Den genererer og persisterer en unik anonym identitet pr. bruger pr. fællesskab - bestående af et dyre-emoji, et autogenereret displaynavn (f.eks. "Anonymous Fox 42") og en baggrundsfarve. En Composite Unique Constraint på [`userId`, `communityId`] sikrer nøjagtigt én anonym identitet pr. bruger pr. fællesskab (16). Denne model er det tekniske fundament for, at brugere kan dele sårbart indhold anonymt i fællesskaber, mens moderatører og administratorer bevarer synlighed til den faktiske afsender.

`CommunityBan` og `CommunityMute` håndterer henholdsvis fællesskabsspecifik udelukkelse og dæmpning af publiceringsrettigheder. Begge modeller indeholder et nullable `expiresAt`-felt til midlertidige sanktioner og en Composite Unique Constraint på [`communityId`, `userId`] for at forhindre dupliserede sanktioner (16).

`HiddenCommunity` tillader en bruger personligt at skjule et fællesskab fra sit eget feed og overblik uden at melde sig ud og uden at påvirke fællesskabets globale `isActive`-tilstand. `BlockedCommunity` er en stærkere variant, der permanent blokerer et fællesskab fra brugerens perspektiv. Begge er koblingstabeller med sammensat primærnøgle og `ON DELETE CASCADE` på begge fremmednøgler.



Figur 4d (Sub-figur C): ER-diagram for fællesskabsdomænet. Viser **Community**-modellen med alle felter samt de tilknyttede adgangsstyringsmodeller: **CommunityMember** (med rollebaseret adgang via **CommunityRole**-enum), **CommunityAnonymousIdentity** (anonyme profiler per fællesskab), **CommunityBan**, **CommunityMute**, **HiddenCommunity** og **BlockedCommunity**.

3.1.4. Sub-figur D: Indhold: Post og Comment

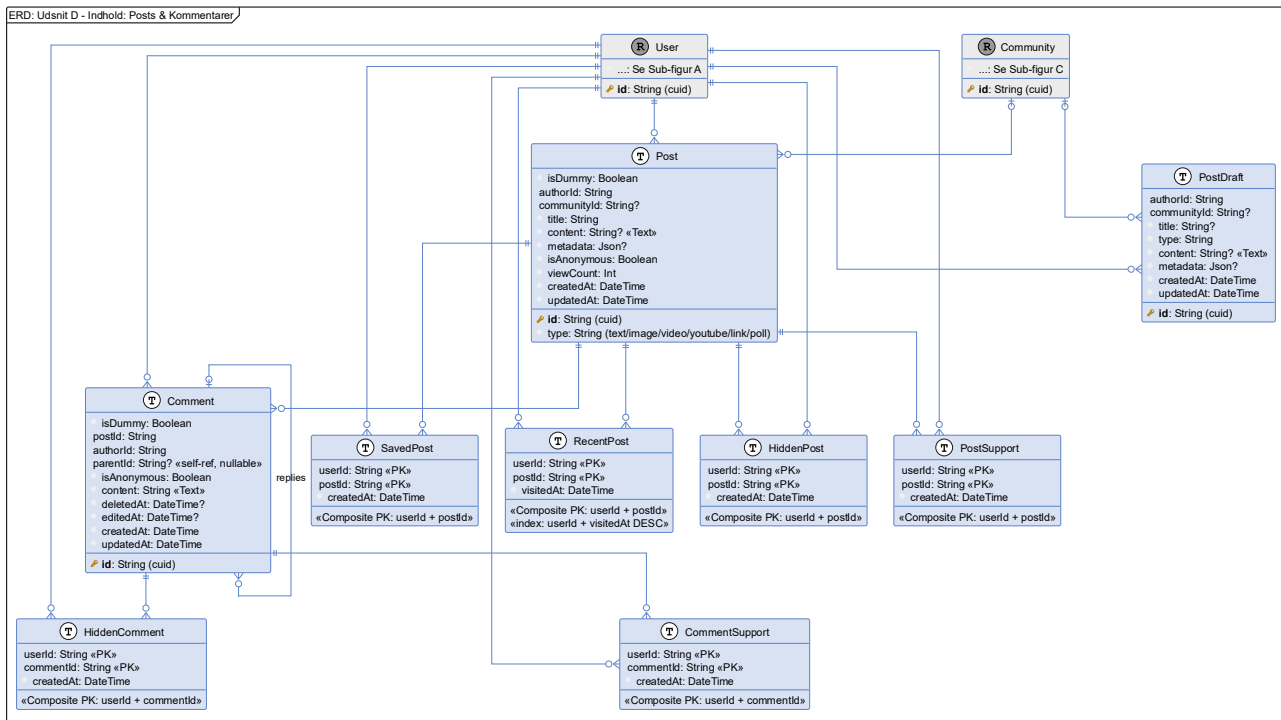
Dette udsnit af ER-diagrammet, illustreret i Figur 4e, beskriver den del af datamodellen, der understøtter indholdsproduktion og brugerinteraktion med indhold.

Post repræsenterer det primære indhold på platformen og understøtter seks indholdstyper via `type`-feltet: `text`, `image`, `video`, `youtube`, `link` og `poll`. Typespecifik metadata (f.eks. `billedeURL`-arrays, afstemningsvalg, ...) lagres fleksibelt i et `metadata`-felt af typen JSON - en tilgang der afvejer fleksibilitet mod stærk typesikkerhed (21). `communityId` er nullable, hvilket muliggør, at et opslag enten tilhører et fællesskab eller publiceres direkte på en brugers profil (16). `isAnonymous`-flaget styrer forfatterens synlighed i frontend-lagets præsentation, mens den relationelle integritet til `User` bevares på databaseniveau.

`PostDraft` fungerer som et midlertidigt arbejdsområde under indholdsoprettelse. Strukturen afspejler `Post` med nullable felter for titel, indhold og fællesskabstilknytning. `ON DELETE CASCADE` på `authorId`-relationen sikrer, at kladder fjernes automatisk ved kontosletning.

Seks koblingstabeller modellerer brugerens interaktioner med opslag: `SavedPost` (gemte opslag), `HiddenPost` (personlig feed-filtrering), `PostSupport` (like/støtte-mekanisme), `RecentPost` (senest besøgte opslag med indekseret `visitedAt`-kolonne for effektiv kronologisk forespørgsel), `CommentSupport` og `HiddenComment`. Alle seks anvender Composite PKs og `ON DELETE CASCADE` på begge fremmednøgler.

`Comment` muliggør asynkron diskussion på opslag. Selvreferencen via det nullable `parentId`-felt implementerer ét indlejningsniveau af tråde (replies), som defineret som en designrestriktion i kravspecifikationen (22) for at undgå databaserekursion. `deletedAt`-kolonnen tillader blød sletning af kommentarer, så trådstrukturen bevares visuelt, selv når det specifikke indhold fjernes. `ON DELETE CASCADE` på `postId` sikrer, at en hel kommentartråd fjernes, når det tilknyttede opslag slettes.



Figur 4e (Sub-figur D): ER-diagram for indholdsdomænet. Viser **Post**- og **Comment**-modellerne med fuld feltbeskrivelse, herunder **parentId**-selvreferencen der muliggør ét niveau af kommentarindlejring. Desuden vises alle junction-modeller for brugerinteraktion med indhold: **PostDraft**, **SavedPost**, **HiddenPost**, **PostSupport**, **RecentPost**, **CommentSupport** og **HiddenComment**.

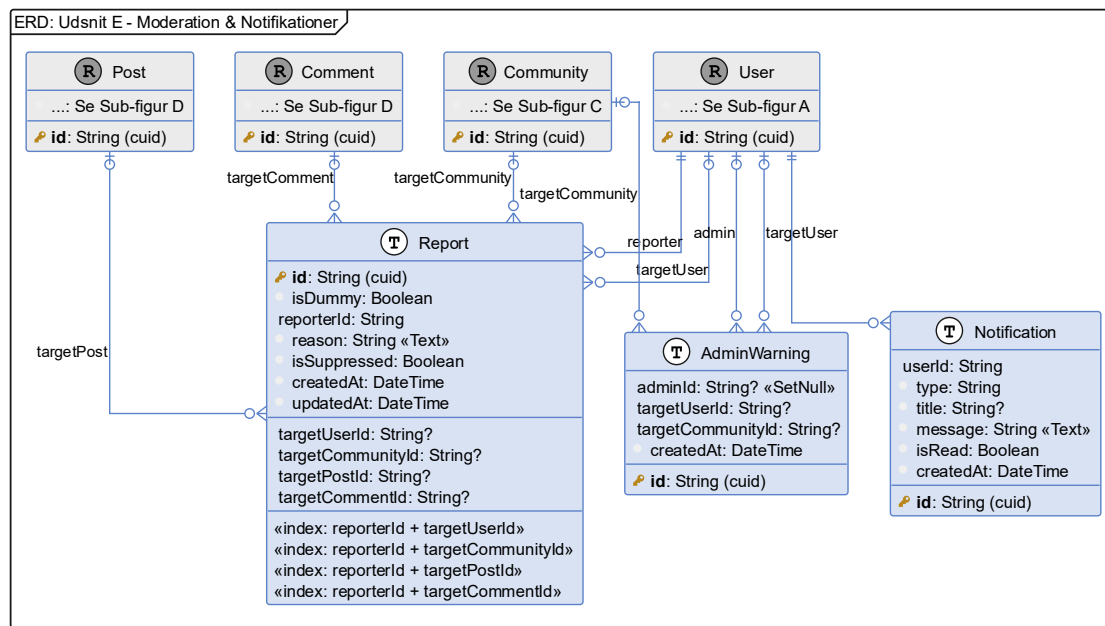
3.1.5. Sub-figur E: Moderation og notifikationer

Dette udsnit af ER-diagrammet, illustreret i Figur 4f, beskriver det moderationsapparat og notifikationssystem, der understøtter platformens tilsynsansvar og brugerinformation.

`Report` implementerer et polymorfisk rapporteringsmønster med fire nullable fremmednøgler - `targetUserId`, `targetCommunityId`, `targetPostId` og `targetCommentId` - hvoraf præcis én er udfyldt pr. rapport afhængig af rapporteringens mål. Hvert indeks er specificeret på databaseniveau for at optimere administratorers forespørgsler på tværs af rapporteringsmål. `isSuppressed`-flaget giver administratorer mulighed for at markere duplikerede eller uberettigede rapporter uden at slette dem, så revisionssporet bevares i overensstemmelse med god praksis for systemadministration.

`AdminWarning` persisterer advarsler udstedt af administratorer til enten brugere eller fællesskaber. `adminId` anvender `onDelete: SetNull`, så advarslen bevares som historisk dokumentation, selvom den udstedende administrator efterfølgende fjernes fra systemet.

`Notification` er systemets mekanisme for direkte kommunikation med den individuelle bruger. Modellen rummer et `type`-felt (f.eks. `WARNING`, `BANNED_ALERT`, `SYSTEM`) til programmatisk routing i frontend-laget. Notifikationer fjernes eksplicit af brugeren via en dedikeret sletningshandling. I MVP-implementeringen styres notifikationsindikatoren i brugerfladen udelukkende af, om `notifications.length > 0` - `isRead`-feltet benyttes således ikke aktivt i UI-laget i denne version, men er bibeholdt i Prisma-skemaet med henblik på fremtidig udvidelse af notifikationssystemet.



Figur 4f (Sub-figur E): ER-diagram for moderations- og notifikationsdomænet. Viser *Report*-modellen med dens fire nullable fremmednøgler til mulige rapporteringsmål (*User*, *Community*, *Post*, *Comment*), *AdminWarning* med nullable referencer til både udsteder og modtager, samt *Notification*-modellen til systemmeddelelser til brugere.

3.1.6. Sub-figur F: Chat, minispil og AI-assistent

Dette udsnit af ER-diagrammet, illustreret i Figur 4g, beskriver datamodellen for platformens kommunikations-, minispil- og AI-funktionalitet.

`DailyStatus` repræsenterer brugerens registrerede energiniveau for en given dag (dagsform) og udgør systemets centrale adaptive tilstandsmodel, som defineret i kravspecifikationen (22). `date`-kolonnen er af typen `Date` (ikke `DateTime`) for at sikre præcis databaseret styring. En Composite Unique Constraint på `[userId, date]` håndhæver forretningsreglen om maksimalt én registrering pr. bruger pr. døgn. Relationen mellem `Message` og `DailyStatus` anvendes på beskedsiden i dagsform-karrusellen: når en bruger besvarer en vens dagsform-note, sendes beskeden med en reference til den pågældende dagsform, så modtageren kan se hvilken note beskeden knytter sig til.

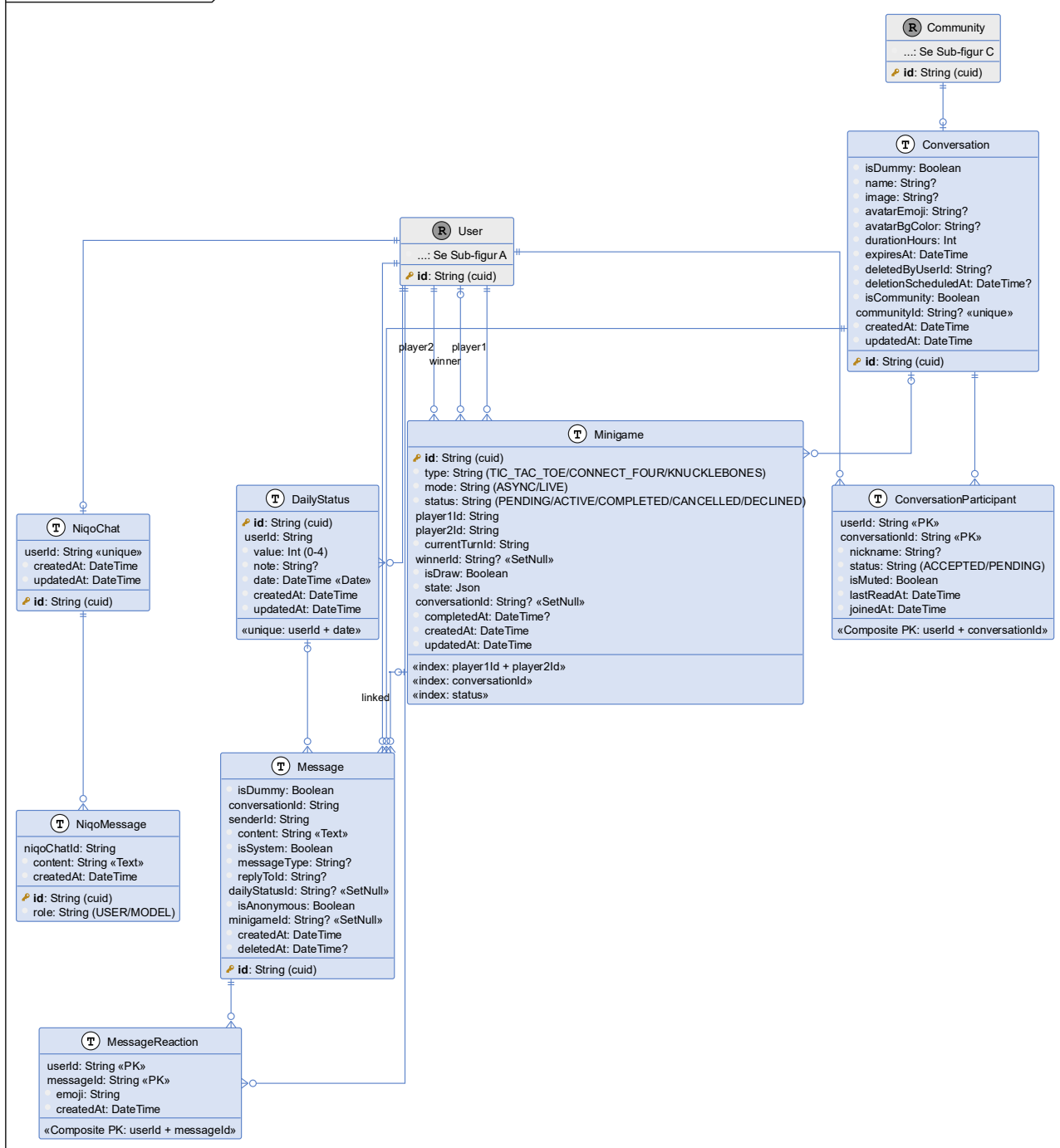
`Conversation` udgør rammen for platformens tidsbegrænsede chats. `expiresAt`-feltet implementerer den planlagte udløbsmekanisme, der er central for platformens design om kortlivede, fortrolige samtaler. `isCommunity: Boolean` og `communityId` (med `@unique-constraint`) muliggør én dedikeret fællesskabschat pr. `Community`. `ConversationParticipant` er koblingstabellen med sammensat primærnøgle, der administrerer deltagerens tilstand (`status`), beskedspærring (`isMuted`) og læsestatus (`lastReadAt`).

`Message` lagrer selve beskedindholdet med mulighed for blød sletning via `deletedAt`, anonyme afsendere via `isAnonymous`, beskedreferencer via `replyToId` og systemgenererede beskeder via `isSystem` + `messageType`. `MessageReaction` er koblingstabellen for emoji-reaktioner med sammensat primærnøgle (`userId` + `messageId`), der håndhæver reglen om én aktiv reaktion pr. bruger pr. besked.

`NiqoChat` er en 1:1-relation til `User`, der fungerer som kontekstbeholder for AI-støttebottens samtalerhistorik. Tilhørende beskeder lagres permanent i `NiqoMessage` i PostgreSQL - én række pr. tur med `role`-feltet (`USER` / `MODEL`) til at skelne afsendertype. Denne persistente lagringsmodel muliggør vedvarende samtalekontekst, der bevares på tværs af sessioner.

Minigame understøtter tre spil typer (TIC_TAC_TOE, CONNECT_FOUR, KNUCKLEBONES) i to tilstande (ASYNC for asynkront spil, LIVE for realtidsbaseret spil). Spiltilstanden lagres fleksibelt i et state-felt af typen Json, da strukturen varierer pr. spiltype. conversationId-relationen anvender onDelete: SetNull, så spilregistreringen persisterer historisk, selv hvis den tilknyttede samtale udløber og slettes. En direkte relation fra Message til Minigame via minigameId muliggør, at systemgenererede beskeder i chat-vinduet kan referere til og kontekstualisere spilinvitationer og resultater.

ERD: Udsnit F - Chat, Spil & AI-assistent



Figur 4g (Sub-figur F): ER-diagram for kommunikationsdomænet. Viser *Conversation* med tilknyttet *ConversationParticipant* og *Message*, *MessageReaction*, samt *DailyStatus*-modellen der kan vedhæftes beskeder. Desuden vises *MiniGame*-modellen (understøtter *TIC_TAC_TOE*, *CONNECT_FOUR* og *KNUCKLEBONES* i *ASYNC* / *LIVE*-tilstand) med dens relation til samtaler og beskeder, samt *NiqoChat* og *NiqoMessage* der udgør AI-assistentens beskedhistorik.

3.2. Data Dictionary (tabelspecifikation)

For at sikre overensstemmelse mellem databasen og forretningsreglerne defineres kritiske designbeslutninger og constraints for samtlige 40 tabeller i Tabel 2. Et centralt arkitektonisk valg er brugen af string-baserede cuid (Collision Resistant Unique Identifiers) som primærnøgler frem for auto-incrementerende integers (23). Dette øger sikkerheden ved at gøre ID'er uforudsigelige - og derved forhindre ID-enumeration angreb - og letter eventuel fremtidig distribuering af databasen (24). Koblelingstabeller uden behov for en selvstændig identifikator benytter i stedet en sammensat primærnøgle (Composite PK) af de to fremmednøgler, der definerer relationen (21).

Tabel 2: Data Dictionary - specifikation af alle 40 tabeller, deres primære ansvarsområde samt kritiske referentielle constraints (regler for sletning).

Data Dictionary		
Tabelnavn	Funktion og forretningslogik	Dataintegritet og constraints (Prisma)
User	Central identitetsstyring. Opbevarer loginoplysninger, rettigheder (Global-Role-enum: USER / ADMIN), profildata og sundhedsoplysninger. Rummer personlige indstillinger for hurtig-reaktioner i chats samt flag for synlighed for sundhedsdata. Kernen i systemet - alt indhold er knyttet til en brugeridentitet.	Unique constraint på email. username er ligeledes unique. ON DELETE CASCADE på al personhenførbart data (jf. GDPR "Right to be forgotten" (25)).
Account	Teknisk Auth.js-tabel. Lagrer forbindelsen til en ekstern OAuth-udbyder (f.eks. Google) via provider og providerAccountId (7, 20).	Composite Unique Constraint på [provider, providerAccountId]. ON DELETE CASCADE ved sletning af den tilhørende User.
Session	Teknisk Auth.js-tabel. Administrerer aktive bruger-sessioner med tilhørende udløbstidspunkt (7).	Unique constraint på sessionToken. ON DELETE CASCADE ved sletning af den tilhørende User.

SignupVerificationToken	Håndterer e-mailbekræftelse ved oprettelse af en credentials-konto. Genererer et tidsbegrænset, unikt token, der sendes til brugerens e-mail-adresse for at bekræfte ejerskab inden aktivering.	Unique constraint på <code>userId</code> (ét aktivt token pr. bruger ad gangen). Unique constraint på <code>token</code>. ON DELETE CASCADE via <code>userId</code>.
PasswordResetToken	Teknisk hjælpetabel til håndtering af adgangskodenulstilling. Genererer et tidsbegrænset, unikt token knyttet til en e-mailadresse.	Unique constraint på <code>token</code>. Composite Unique Constraint på <code>[email, token]</code>. ON DELETE CASCADE via <code>userId</code>.
AccountDeletionToken	Håndterer bekræftelse af permanent kontosletning. Genererer et tidsbegrænset token, der sendes til brugerens e-mail for at validere intentionen inden uigenkaldelig sletning.	Unique constraint på <code>token</code>. Composite Unique Constraint på <code>[email, token]</code>. ON DELETE CASCADE via <code>userId</code>.
EmailVerificationToken	Håndterer e-mailbekræftelse ved ændring af brugerens e-mailadresse. Sikrer ejerskabsbekræftelse af den nye adresse inden opdatering.	Unique constraint på <code>token</code>. Composite Unique Constraint på <code>[email, token]</code>. ON DELETE CASCADE via <code>userId</code>.

UsernameChangeToken	Håndterer det sikrede flow ved ændring af brugernavn, herunder bekræftelse via e-mail. Indeholder det ønskede <code>newUsername</code> som del af flowet, og understøtter den 30-dages spærreperiode via <code>usernameChangedAt</code> på <code>User</code>.	Unique constraint på <code>token</code>. Composite Unique Constraint på <code>[email, token]</code>. ON DELETE CASCADE via <code>userId</code>.
GlobalBan	Bandlyser en bruger på platformniveau. Designet til at persistere som en registrering, selv efter den tilknyttede konto slettes - f.eks. for at forhindre gentilmelding via samme e-mail.	Unique constraint på <code>email</code> (det permanente identifikationspunkt). <code>userId</code>-relationen anvender <code>onDelete: SetNull</code> - den bandlystes e-mail bevares, men brugertilknytningen nulstilles ved kontosletning. Unique constraint på det nullable <code>deleteToken</code>-felt.
GlobalMute	Dæmper en brugers publiceringsrettigheder på platformniveau midlertidigt. <code>expiresAt</code>-feltet muliggør tidsbegrænsede sanktioner.	Unique constraint på <code>userId</code> (én aktiv global mute pr. bruger). ON DELETE CASCADE ved kontosletning.
GlobalBlock	Koblingstabel der registrerer en brugers aktive blokering af en anden bruger på platformniveau. Håndhæves i feed-filttering og beskedrestriktioner.	Composite PK på <code>[blockerId, blockedId]</code>. ON DELETE CASCADE på begge fremmednøgler.

Friendship	Repræsenterer en etableret, bidirektionel venskabsrelation mellem to brugere. Muliggør deling af privat indhold og beskeder afhængig af brugerens messagingPermission-indstilling.	Composite Unique Constraint på [userId, friendId] forhindrer duplikerede relationer. ON DELETE CASCADE på begge fremmednøgler.
FriendRequest	Modellerer den afventende venskabsanmodning og styrer den asynkrone godkendelseslogik via statusfeltet (PENDING, ACCEPTED, DECLINED).	Composite Unique Constraint på [senderId, receiverId] forhindrer adskillige samtidige anmodninger. ON DELETE CASCADE på begge fremmednøgler.
Community	Definerer de logiske fællesskaber, som brugere samles i. Rummer konfiguration for synlighed (isPrivate), aktivitetsstatus (isActive) og kategori. Understøtter administrativ suspension via banReason og bannedUntil.	Unique constraint på name. Slettes et fællesskab, slettes alle tilhørende Post, Comment og CommunityMember via ON DELETE CASCADE.
CommunityMember	Koblingstabel der modellerer N:M-relationen mellem User og Community. Indeholder brugerens rolle (CommunityRole: USER, MODERATOR, ADMIN, OWNER) og medlemsstatus (ACCEPTED / PENDING).	Composite PK på [userId, communityId] sikrer, at en bruger kun kan optræde én gang som medlem i hvert fællesskab. ON DELETE CASCADE på begge fremmednøgler.

CommunityAnonymousIdentity	Persisterer en unik anonym identitet pr. bruger pr. fællesskab (dyre-emoji, autogenereret displaynavn, baggrundsfarve). Udgør det tekniske fundament for platformens anonyme publiceringsmulighed, der lader brugere dele sårbart indhold uden at afsløre deres rigtige identitet for øvrige medlemmer.	Composite Unique Constraint på [userId, communityId] sikrer nøjagtigt én anonym identitet pr. bruger pr. fællesskab. ON DELETE CASCADE på begge fremmednøgler.
CommunityBan	Udelukker en specifik bruger fra et specifikt fællesskab. expiresAt-feltet muliggør midlertidige udelukkelse.	Composite Unique Constraint på [communityId, userId] forhindrer duplikerede sanktioner. ON DELETE CASCADE på begge fremmednøgler.
CommunityMute	Dæmper en brugers publiceringsrettigheder i et specifikt fællesskab midlertidigt. expiresAt-feltet muliggør tidsbegrænsede sanktioner.	Composite Unique Constraint på [communityId, userId]. ON DELETE CASCADE på begge fremmednøgler.
HiddenCommunity	Tillader en bruger personligt at skjule et fællesskab fra sit eget overblik og feed, uden at melde sig ud og uden at påvirke fællesskabets globale isActive-tilstand.	Composite PK på [userId, communityId] sikrer, at hvert fællesskab kun kan skjules én gang pr. bruger. ON DELETE CASCADE på begge fremmednøgler.

BlockedCommunity	Registrerer en brugers permanente blokering af et fællesskab, der forhindrer fællesskabet i at optræde i brugerens feed eller søgeresultater.	Composite PK på [userId, communityId]. ON DELETE CASCADE på begge fremmednøgler.
Post	Repræsenterer det primære indhold på platformen. Understøtter seks indholdstyper (text, image, video, youtube, link, poll). Typespecifik metadata lagres i metadata: json. communityId er nullable, da et opslag kan tilhøre enten et fællesskab eller brugers egen profil. isAnonymous-flaget styrer forfatterens synlighed i frontend-præsentationen, mens den relationelle integritet til User bevares i databasen.	Kræver authorId. communityId er nullable. ON DELETE CASCADE på authorId- og communityId-relationen - opslaget slettes, hvis forfatter eller fællesskab forsvinder.

PostDraft	Opbevarer brugerens ufærdige opslag (kladder) som et midlertidigt arbejdsområde under indholdsoprettelse. Kan være tilknyttet et specifikt fællesskab eller brugerens profil (<code>communityId</code> er nullable). Ejerskabsvalidering i Service-laget forhindrer uautoriseret adgang til andres kladder.	<code>communityId</code> er nullable. ON DELETE CASCADE på <code>authorId</code>-relationen sikrer, at kladder slettes automatisk med brugerkontoen.
SavedPost	Giver brugere mulighed for at gemme (bogmærke) opslag til personlig reference.	Composite PK på [<code>userId</code>, <code>postId</code>]. ON DELETE CASCADE på begge fremmednøgler.
HiddenPost	Tillader brugere personligt at skjule specifikke opslag fra deres feed uden at rapportere dem.	Composite PK på [<code>userId</code>, <code>postId</code>]. ON DELETE CASCADE på begge fremmednøgler.
PostSupport	Registrerer en brugers støttereaktioner på et opslag (platformens like / støtte-mekanisme).	Composite PK på [<code>userId</code>, <code>postId</code>] sikrer, at en bruger kun kan støtte et opslag én gang. ON DELETE CASCADE på begge fremmednøgler.
RecentPost	Lagrer en brugers senest besøgte opslag med tidsstempel (<code>visitedAt</code>) til visning af "senest sete opslag"-funktionalitet.	Composite PK på [<code>userId</code>, <code>postId</code>]. Indeks på [<code>userId</code>, <code>visitedAt</code> DESC] for effektiv kronologisk forespørgsel. ON DELETE CASCADE på begge fremmednøgler.

Comment	Muliggør asynkron diskussion på opslag. Selvreference via det nullable <code>parentId</code> implementerer ét indlejningsniveau af tråde, som defineret som en designrestriktion i kravspecifikationen (22). <code>deletedAt</code> muliggør blød sletning, så trådstrukturen bevares visuelt.	<code>parentId</code> er nullable. ON DELETE CASCADE på <code>postId</code> sikrer, at tråden fjernes med opslaget. ON DELETE CASCADE på <code>authorId</code>.
CommentSupport	Registrerer en brugers støttereaktion på en specifik kommentar.	Composite PK på [<code>userId</code>, <code>commentId</code>]. ON DELETE CASCADE på begge fremmednøgler.
HiddenComment	Tillader brugere personligt at skjule specifikke kommentarer fra deres visning.	Composite PK på [<code>userId</code>, <code>commentId</code>]. ON DELETE CASCADE på begge fremmednøgler.
Conversation	Definerer rammen for platformens tidsbegrænsede chats. <code>expiresAt</code> implementerer den planlagte udløbsmekanisme. <code>isCommunity</code>-flag kombineret med <code>communityId</code> (unique) muliggør én dedikeret fællesskabschat pr. Community. Understøtter 1:1, gruppe- og fællesskabsbaserede samtaler i ét samlet design.	<code>communityId</code> er nullable med @unique-constraint. <code>deletionScheduledAt</code> markerer, hvornår et cron-job kan fjerne udløbet data. ON DELETE CASCADE via <code>communityId</code>.

ConversationParticipant	Koblingstabel der administrerer deltagerens tilstand (ACCEPTED / PENDING), beskedspærring (isMuted) og læsestatus (lastReadAt) i en samtale.	Composite PK på [userId, conversationId]. ON DELETE CASCADE på begge fremmednøgler.
Message	Indeholder selve beskedindholdet i en samtale. Understøtter blød sletning (deletedAt), anonyme afsendere (isAnonymous), beskedreferencer (replyToId), systemgenererede beskeder (isSystem, messageType) samt forbindelser til en dagsform (dailyStatusId) og et spil (minigameId).	ON DELETE CASCADE på senderId og conversationId. dailyStatusId og minigameId anvender onDelete: SetNull for at bevare beskedhistorik, selv hvis den tilknyttede dagsform- eller spilpost fjernes.
MessageReaction	Koblingstabel der håndterer en brugers emoji-reaktion på en specifik besked. Sikrer, at en bruger kan udtrykke én følelse pr. besked asynkront.	Composite PK på [userId, messageId] sikrer én aktiv reaktion pr. bruger pr. besked. ON DELETE CASCADE på begge fremmednøgler.
DailyStatus	Historik over brugerens daglige energiniveau (dagsform). value-feltet (0-4, nulindekseret) matcher den 1-5-skala, der præsenteres i brugerfladen. date-kolonnen er af typen Date (ikke DateTime) for præcis datobaseret styring.	Composite Unique Constraint på [userId, date] håndhæver forretningsreglen om maksimalt én registrering pr. bruger pr. døgn. ON DELETE CASCADE via userId.

Report	Modellerer et polymorfisk rapporteringsmønster, hvor én rapport kan pege på et af fire mulige mål: User, Community, Post eller Comment. Kun én af de fire nullable fremmednøgler er udfyldt pr. rapport. isSuppressed giver administratorer mulighed for at markere duplikerede rapporter uden at slette dem.	Fire nullable FK'er med separate indeks for effektiv administratorforespørgsel. ON DELETE CASCADE på reporterId og samtlige nullable mål-FK'er.
AdminWarning	Persisterer advarsler udstedt af administratorer til enten brugere (targetUserId) eller fællesskaber (targetCommunityId). Bevares som historisk dokumentation selv ved efterfølgende kontosletning.	adminId anvender onDelete: SetNull - advarslen bevares, selvom den udstedende administrator slettes. ON DELETE CASCADE på targetUserId og targetCommunityId.

Notification	Systemets mekanisme for direkte kommunikation med den individuelle bruger. type-feltet (f.eks. WARNING, BANNED_ALERT, SYSTEM) muliggør programmatisk routing i frontend-laget. isRead-feltet er defineret i Prisma-skemaet til fremtidig implementering. I den nuværende MVP-version styres notifikationsindikatoren udelukkende af, om listen er ikke-tom - der er ingen aktiv markér-som-læst-mekanisme i UI-laget.	ON DELETE CASCADE via userId.
NiqoChat	1:1-relation til User der fungerer som kontekstbeholder for AI-støttebottens (Niqo) samtalerhistorik. Persisteret i PostgreSQL for at opretholde vedvarende samtalekontekst på tværs af sessioner.	Unique constraint på userId (én Niqo-chat pr. bruger). ON DELETE CASCADE via userId.
NiqoMessage	Lagrer de individuelle beskeder i en NiqoChat-session. role-feltet (USER / MODEL) skelner mellem brugerens input og AI-modellens svar.	ON DELETE CASCADE via niqoChatId.

Minigame	Understøtter tre spiltyper (TIC_TAC_TOE, CONNECT_FOUR, KNUCKLEBONES) i to tilstande (ASYNC / LIVE). Spiltilstanden lagres fleksibelt i state: Json. En relation til Conversation via conversationId knytter spillet til en chat, men onDelete: SetNull sikrer, at spilregistreringen persisterer, selv hvis samtalen udløber.	conversationId er nullable med onDelete: SetNull. winnerId anvender onDelete: SetNull. Indeks på [player1Id, player2Id], [conversationId] og [status].
----------	--	---

4. Applikationsmodel (softwaredesign)

Dette afsnit beskriver overgangen fra de overordnede arkitektur- og databasemodeller til det konkrete softwaredesign. Applikationsmodellen fungerer som byggevejledningen for kildekoden og fastlægger, hvordan forretningslogik, dataudveksling og adgangskontrol struktureres i applikationen.

4.1. Designmønstre og arkitektoniske valg

For at sikre høj vedligeholdelsesvenlighed (Supportability) og lav kobling (Low Coupling) mellem systemets moduler, er Next.js applikationen implementeret ud fra et flerlagret arkitekturmønster (Layered Architecture) (12). Fordelingen af ansvar er opsummeret i nedenstående Tabel 3:

Tabel 3: Oversigt over systemets flerlagshierarki. Tabellen beskriver ansvarsfordelingen mellem de enkelte lag i arkitekturen samt de dertilhørende tekniske fordele.

Systemets flerlagshierarki (Layered Architecture)		
Arkitektonisk lag	Ansvar og rolle	Tekniske fordele
Route Handlers / Server Actions (Controller)	Håndterer indgående HTTP-forespørgsler (Request/Response) og parameter-validering. Fungerer som trafikkontrol.	Ved at fjerne forretningslogik herfra holdes laget tyndt og fokuseret på routing.
Service-laget	Indkapsler systemets kerneforretningslogik (f.eks. beregning af dagsformstendenser).	Muliggør genbrug af logik på tværs af systemet og tillader isoleret unit-testing uden behov for at simulere HTTP-requests.
DTO-laget (Zod) (14, 15)	Definerer en stærk kontrakt for dataudveksling (f.eks. <code>Create-DailyStatusDTO</code>).	Sikrer typesikkerhed og forhindrer over-posting (en sikkerhedsrisiko, hvor en bruger forsøger at sende uautoriserede felter, som f.eks. <code>role: ADMIN</code> , med i en forespørgsel).
Data Access-laget	Fungerer som et abstraktionsslag over PostgreSQL-databasen via Prisma ORM.	Et forenklet mellemlag, der gør det muligt at interagere med databasen via typesikker kode i stedet for fejlbehæftet, rå SQL-kode.

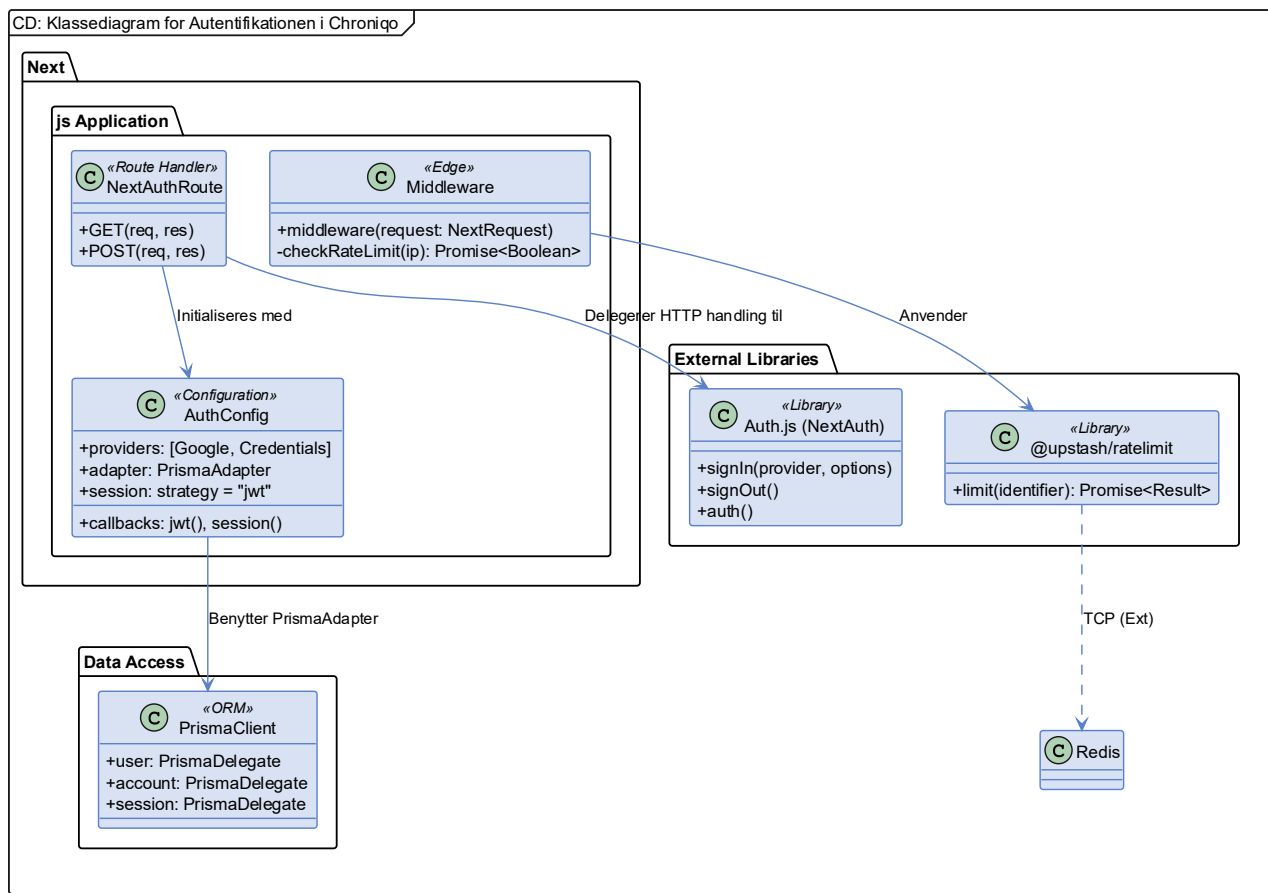
4.2. Design klassediagram (UML)

Da systemet dækker over alt fra kompleks sikkerhed til sociale funktioner, asynkron chat og AI-integration, er klassediagrammet opdelt i tre logiske hovedområder for at bevare overblikket: Autentificering og sikkerhed (afsnit 4.2.1), Kernerdomænet: dagsform, indhold og fællesskaber (afsnit 4.2.2), og Kommunikationsdomænet: chat, minispil og AI-assistent (afsnit 4.2.3).

4.2.1. Autentificering og sikkerhed (Auth Domain)

Som det fremgår af Figur 5, illustrerer designet det sikkerhedslag, der forvalter login-processer, sessionsstyring og rettighedstildeling. Designet afspejler en markant arkitektonisk forenkling gennem implementeringen af Auth.js (7), som effektivt indkapsler den komplekse OAuth 2.0-logik (26) og abstraherer protokollens tekniske detaljer væk fra applikationslaget.

Arkitekturen i dette domæne er centreret omkring en høj grad af indkapsling (Encapsulation), hvor Auth.js fungerer som det primære bindeled mellem eksterne identitetsudbydere (f.eks. Google) og systemets interne datamodel. Ved at uddelegere de kryptografiske operationer og state-håndteringen til biblioteket, sikres en stabil og fejlsikker integration, der minimerer risikoen for manuelle fejl i sikkerhedskritiske processer og optimerer systemets samlede vedligeholdelsesvenlighed.

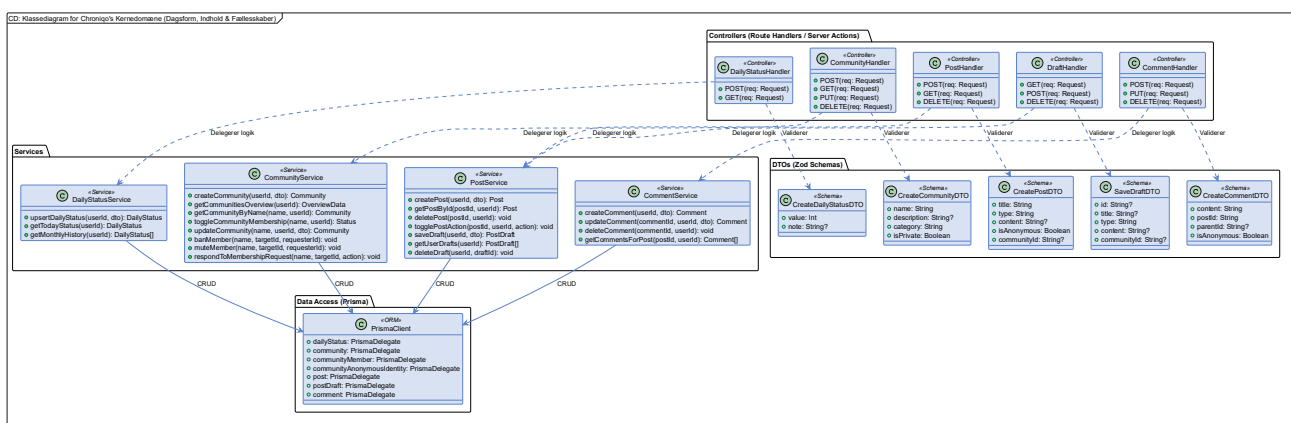


Figur 5: Design-klassediagram for autentificeringsmodulet. Viser integrationen af Auth.js i Next.js og middleware-lagets interaktion med Upstash Redis for rate limiting.

4.2.2. Kernerdomænet (dagsform og socialt)

Som illustreret på Figur 6, udfolder dette domæne systemets primære forretningsværdi gennem de tekniske interaktioner med dagsformer, fællesskaber og opslag. Designet afspejler en moderne Next.js-arkitektur, hvor indgangen til domænet orkestreres via enten Route Handlers (13) eller Server Actions (27). Disse fungerer som systemets controllers, der håndterer den indledende validering af input gennem Zod-baserede DTO'er (Data Transfer Objects) (14, 15), hvilket etablerer en streng kontrakt for dataintegritet mellem klient og server.

Notationsmæssigt skelnes der i diagrammet mellem permanente afhængigheder (fuldt optrukne pile) og flygtige dataoverførsler (stiplede pile). Dette tydeliggør, hvordan DTO-laget udelukkende fungerer som midlertidige beholdere, der sikrer, at kun valideret og typesikre data videresendes til Service-laget. Ved at isolere forretningslogikken her, garanteres en stabil eksekvering, der er uafhængig af, hvorvidt anmodningen oprinder fra et traditionelt API-kald eller en moderne server-side handling.



Figur 6: Design-klassediagram for systemets kernerdomæne. Diagrammet viser lagdelingen fra Controllers til Data Access på tværs af `DailyStatusService`, `CommunityService`, `PostService` og `CommentService`, og markerer forskellen på systemets faste arkitektur (associationer) og de midlertidige datakontrakter i form af DTO'er (dependencies).

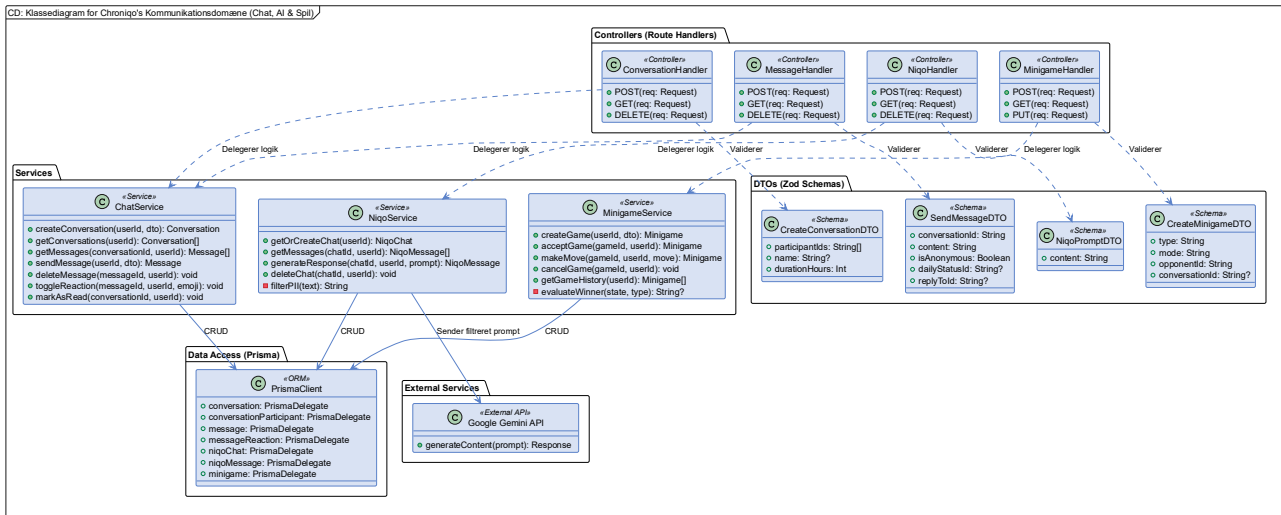
4.2.3. Kommunikationsdomænet (chat, minispil og AI)

Som illustreret i Figur 7, modellerer dette domæne tre tætforbundne services, der tilsammen udgør platformens asynkrone kommunikationsinfrastruktur.

`ChatService` håndterer oprettelse og administration af tidsbegrænsede samtaler (`Conversation`), afsendelse og sletning af beskeder (`Message`) samt emoji-reaktioner via toggle-mekanismen, der er dokumenteret i SD 5.6. Servicen er den eneste indgang til al beskedrelateret persistens og sikrer derved, at forretningsreglerne for samtalers livscyklus og deltagerrettigheder håndhæves konsistent.

`NiqoService` orkestrerer interaktionen med Google Gemini API. Den private `filterPII`-metode udgør det Privacy by Design-skjold, der er beskrevet i SD 5.11: brugerinput renses for personhenførbare oplysninger via Regex, inden prompten videresendes til den eksterne API. Konversationshistorikken gemmes permanent i `NiqoChat` / `NiqoMessage` via Prisma for at opretholde en meningsfuld samtalekontekst på tværs af sessioner.

`MinigameService` styrer hele livscyklussen for platformens minispil - fra invitation (`createGame`) over spiludførelse (`makeMove`) til resultatevaluering (`evaluateWinner`). Den private evalueringsmetode beregner vinder eller uafgjort afhængig af spiltype og opdaterer `Minigame.state` atomisk i databasen.



Figur 7: Design-klassediagram for kommunikationsdomænet. Viser *ChatService*, *NiqoService* og *MinigameService* med deres respektive controllers, DTO-kontrakter og dataadgangslag. Bemærk *NiqoService*'s afhængighed af den eksterne Google Gemini API (28) samt den private *filterPii*-metode, der sikrer Privacy by Design-princippet forud for ekstern API-kommunikation.

5. Systemsekvensdiagrammer (dynamisk adfærd)

Hvor de foregående afsnit har beskrevet systemets statiske opbygning, fokuserer dette afsnit på at efterprøve arkitekturens dynamiske virkemåde. I stedet for en udtømmende gennemgang er der foretaget et strategisk valg af de mest *Architecturally Significant Use Cases*. Ved at analysere disse kritiske flows - fra sikkerhedsprotokoller i Auth.js (7) til den centrale domænelogik - valideres det, at systemets design er både robust og funktionelt.

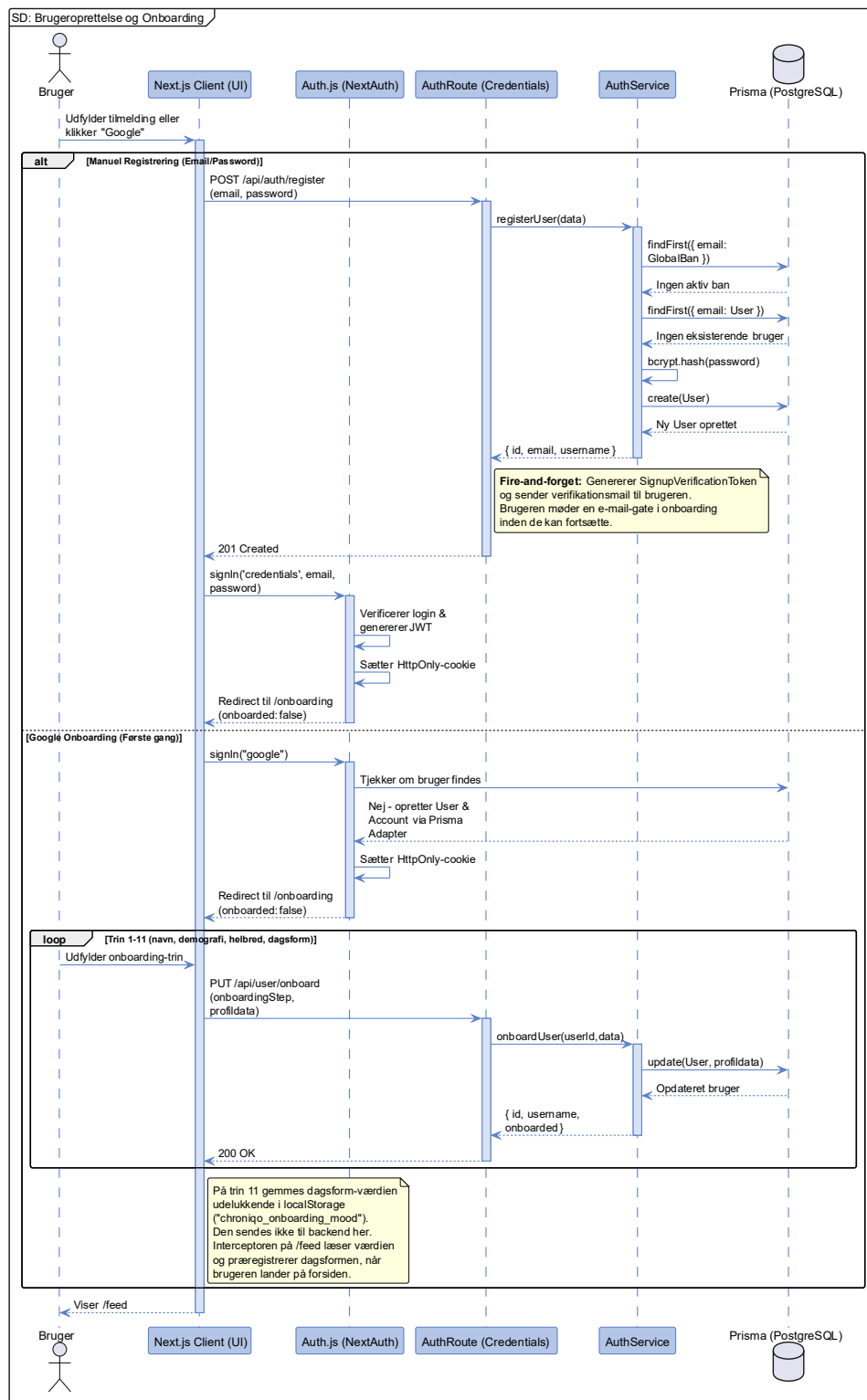
Gennem sekvensdiagrammerne illustreres det, hvordan systemets forskellige lag (Frontend-lag, Next.js Middleware, Route Handlers, Services og persistenslaget) interagerer i praksis for at løse specifikke forretningsopgaver.

Bemærkning til sporbarhed: For at bevare en stærk rød tråd er de enkelte diagrammer knyttet til deres oprindelige User Story (US) fra kravspecifikationen (22). Visse sekvensdiagrammer modellerer dog tekniske sikkerhedskrav (f.eks. NFR: Non-Functional Requirements) eller afledte sub-features (som kladder i afsnit 5.9 og personlig skjulning i afsnit 5.10). Disse er ikke eksplicite User Stories i sig selv, men er designet under udviklingen for at understøtte robustheden og den skånsomme brugeroplevelse i henholdsvis oprettelsen af opslag (US3.8) og administrationen af fællesskaber (US3.14).

5.1. SD: Brugeroprettelse og onboarding (US1.1)

Brugeroprettelse, som er illustreret på Figur 8, viser systemets dynamiske adfærd ved oprettelse af nye konti. Ved hjælp af en Alternative (alt) blok modelleres to scenarier, som sikrer en ensartet og sikker håndtering af nye brugere i systemet:

- **Manuel registrering (Credentials):** `AuthService.registerUser()` tjekker først aktivt om e-mailen er globalt bandlyst via `GlobalBan`-tabellen, dernæst om e-mailen allerede er i brug, hasher adgangskoden med `bcrypt` og opretter brugeren. `Auth.js` gennemfører efterfølgende det første login og omdirigerer til onboarding (7, 29).
- **Google Onboarding (Første gang):** Når en bruger logger ind via Google for første gang, oprettes brugeren automatisk via `Auth.js`' Prisma Adapter. Da `onboarded`-flaget er `false`, omdirigeres brugeren til onboarding-flowet. Dette flow gennemløber trin 1-11 via `PUT /api/user/onboard`, der kalder `AuthService.onboardUser()` og progressivt opdaterer navn, demografiske data og helbredsoplysninger. På trin 10-11 angiver brugeren sin dagsform og en valgfri note - disse værdier sendes ikke til backend, men gemmes udelukkende i `localStorage` under nøglen `chroniqo_onboarding_mood`. Interceptoren på `/feed` læser værdien ved første sideindlæsning og præregistrerer dagsformen, så brugeren møder et forudfyldt registreringsdialogvindue ved ankomst.

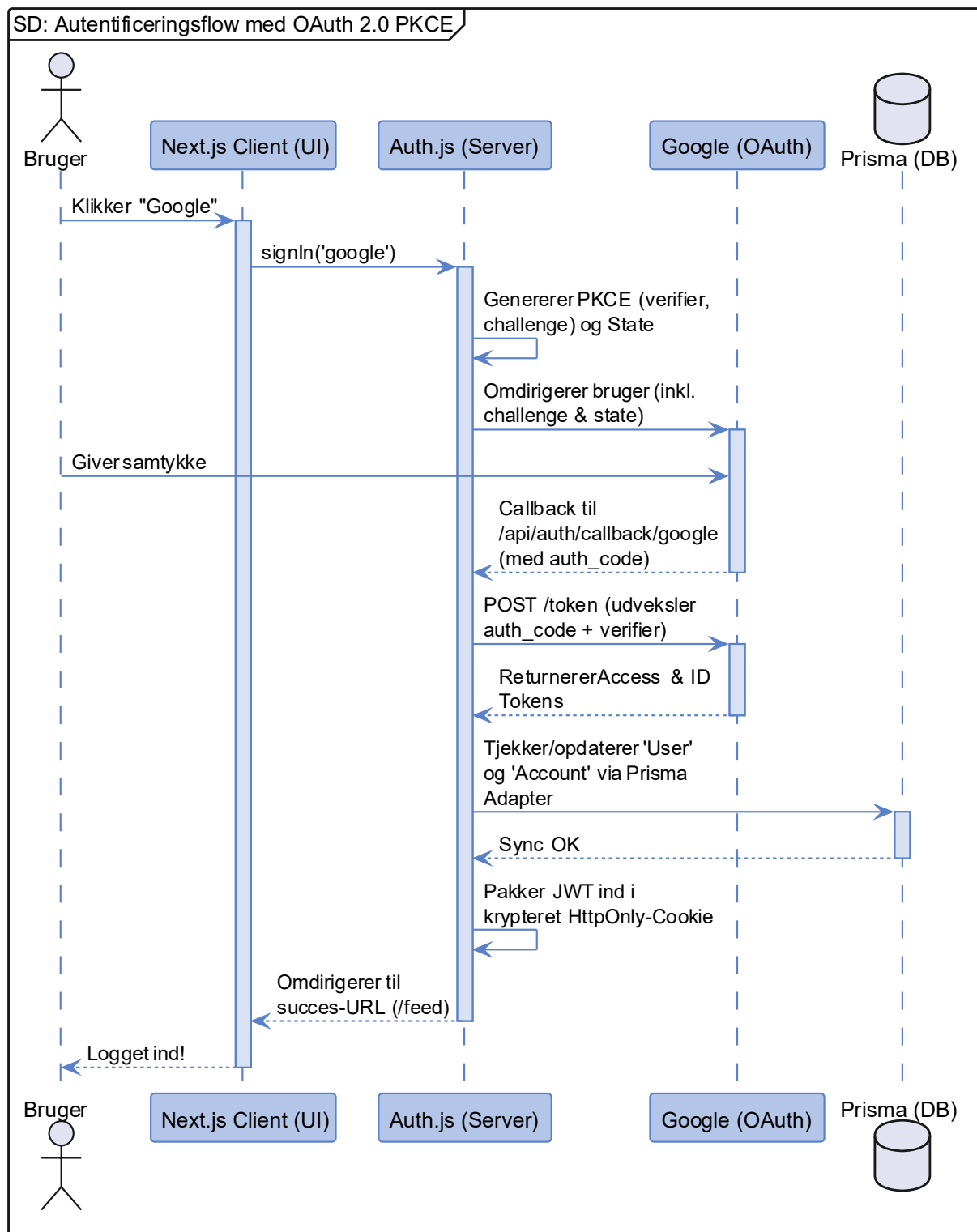


Figur 8: Sekvensdiagram for Brugeroprettelse og Onboarding (US1.1). Viser forskellen i kontrolflowet mellem en manuel credentials-registrering - inklusiv globalt ban-tjek og bcrypt-hashing i AuthService - og fuldførelsen af en Google-onboarding via det trinvis `PUT /api/user/onboard`-flow. Bemærk at dagsform-værdien fra trin 11 gemmes i `LocalStorage` og ikke persisteres til databasen i dette flow - dagsformen registreres i stedet af interceptoren, når brugeren lander på `/feed`.

5.2. SD: Autentificeringsflow med OAuth 2.0 PKCE (US1.2/1.3)

Autentificeringsflowet, som er illustreret på Figur 9, er designet til at håndtere både lokalt login og Single Sign-On (SSO) via Google. Da systemet implementerer Auth.js (7), håndteres kompleksiteten i OAuth 2.0 Authorization Code Flow med PKCE (Proof Key for Code Exchange) (26) som en integreret del af bibliotekets kernefunktionalitet.

Dette minimerer den manuelle implementeringsindsats og sikrer, at udvekslingen af koder og tokens mellem Next.js-serveren og identitetsudbyderen (Google) sker efter gældende sikkerhedsstandarder. Flowet afsluttes med udstedelsen af en krypteret `HttpOnly` session-cookie til klienten (8), hvilket etablerer en sikker og vedvarende session uden eksponering af følsomme tokens i browserens hukommelse.



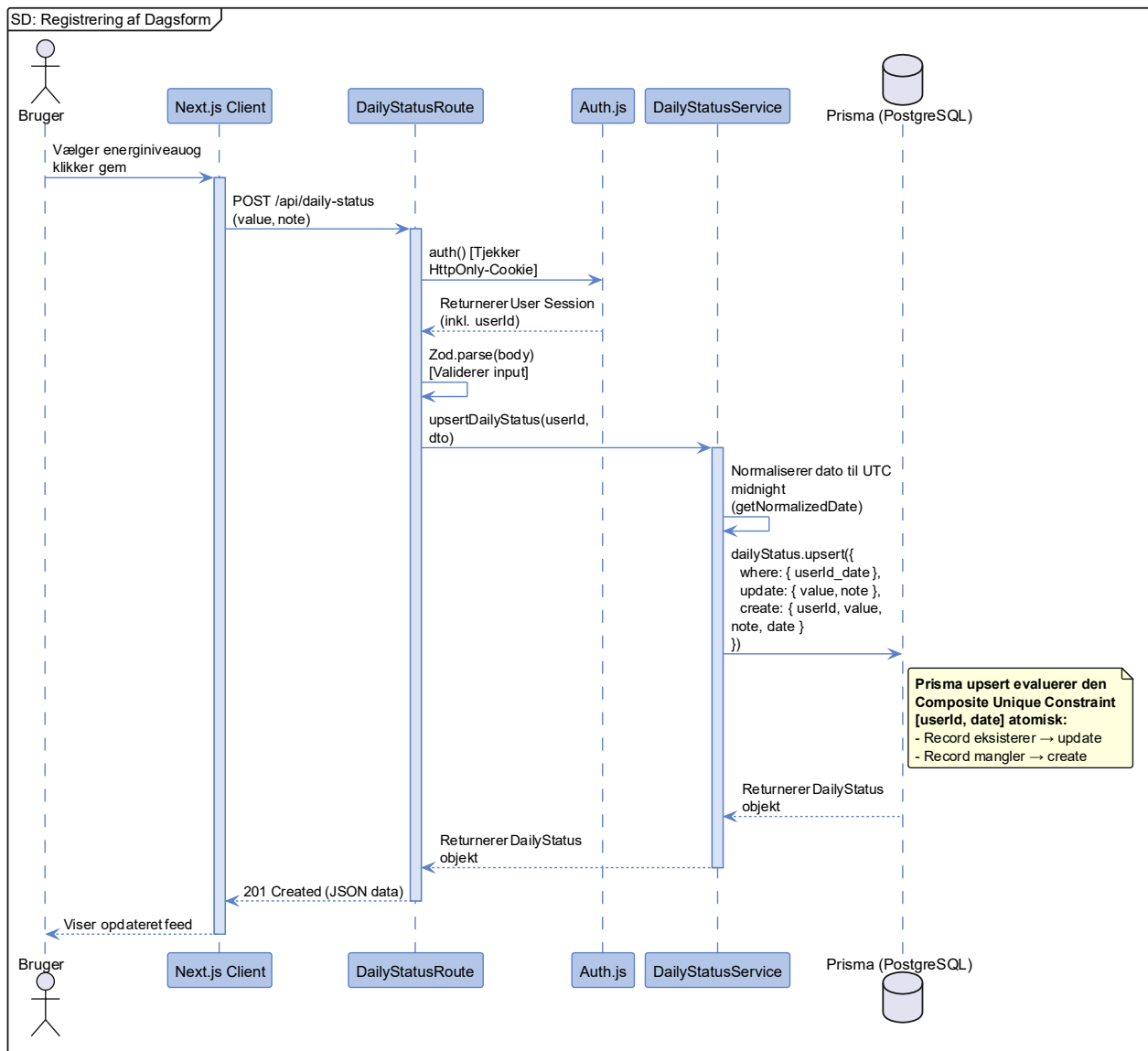
Figur 9: Sekvensdiagram for Autentificeringsflow med OAuth 2.0 PKCE (US1.2/1.3) (Auth.js-abstraktion). Illustrerer hvordan Auth.js orkestrerer OAuth-forhandlingen med Google (inkl. PKCE) og automatisk synkroniserer brugerdata med PostgreSQL-persistenslaget via Prisma.

5.3. SD: Registrering af dagsform (US2.1)

Registreringen af dagsform udgør systemets arkitektoniske hjerte og er det primære omdrejningspunkt for at skabe et adaptivt fællesskab, der tager højde for brugernes aktuelle energiniveauer. Som illustreret på Figur 10, modellerer dette sekvensdiagram det flow, der aktiveres, når en bruger angiver sin status via systemets frontend.

Diagrammet demonstrerer, hvordan Next.js Route Handlers (13) fungerer som den centrale controller, der orkestrerer interaktionen mellem sikkerhed og forretningslogik. Her anvendes Auth.js (7) til at verificere brugerens identitet via den sikre session-cookie på serveren, før inputtet gennemgår en streng validering med Zod-DTO'er (14, 15).

For at sikre dataintegritet og en naturlig brugeroplevelse anvender `DailyStatusService.upsertDailyStatus()` en enkelt, atomisk Prisma `upsert`-operation (30). Prisma evaluerer den Composite Unique Constraint `[userId, date]` internt og afgør automatisk, om en dagsform for den normaliserede UTC-dato skal oprettes eller opdateres - uden en separat `findFirst`-forespørgsel. Dette sikrer en let og ubesværet arbejdsgang for brugeren og eliminerer risikoen for race conditions.



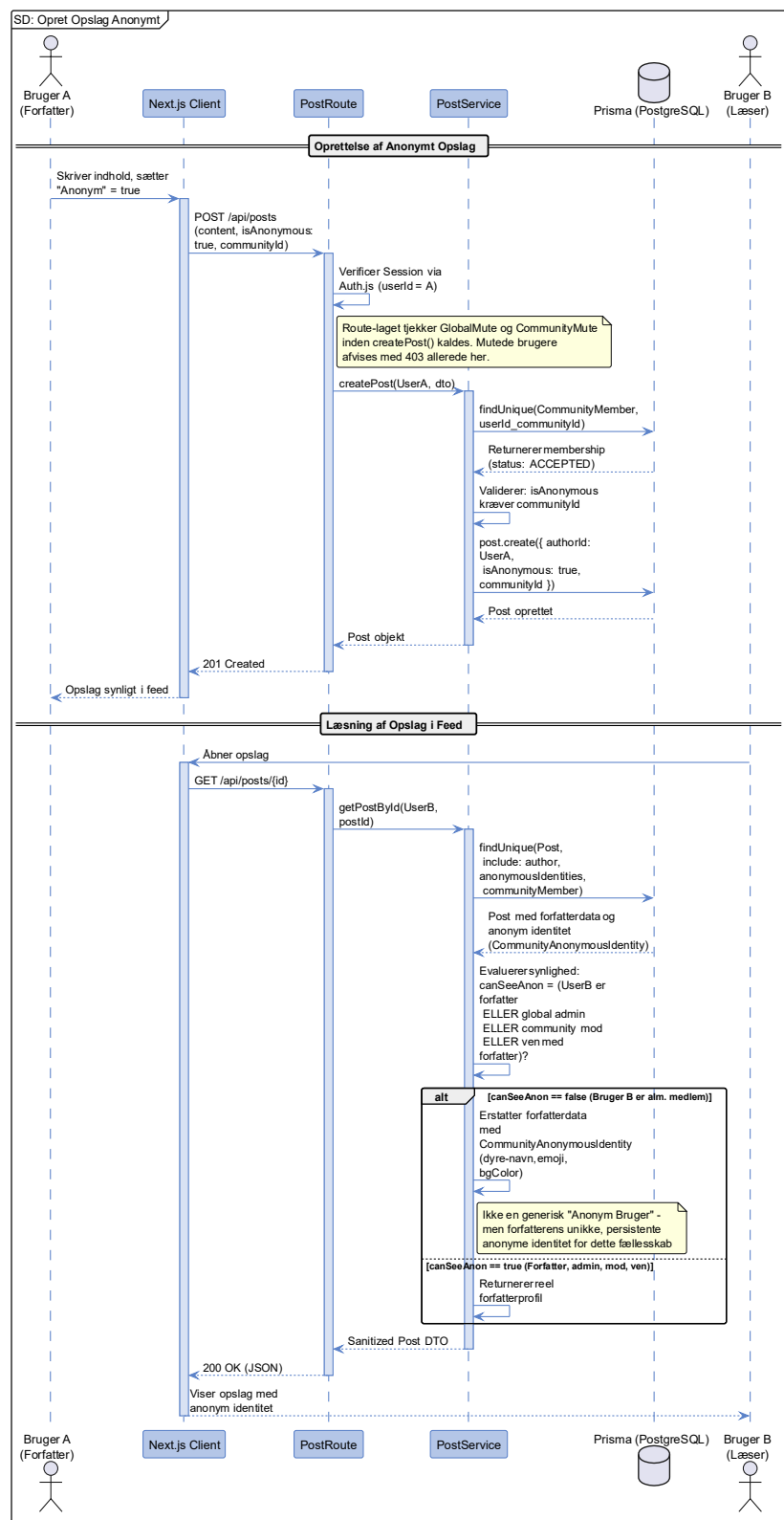
Figur 10: Sekvensdiagram for Registrering af Dagsform (US2.1). Illustrerer anvendelsen af Auth.js sessions-tjek via `HttpOnly` cookies samt en enkelt, atomisk Prisma `upsert`-operation (30), der evaluerer den Composite Unique Constraint `[userId, date]` for at håndhæve forretningsreglen om maksimalt én registrering pr. bruger pr. døgn.

5.4. SD: Opret opslag anonymt (US3.10)

Muligheden for at oprette anonyme opslag er en fundamental funktion for at sikre målgruppens tryghed, når der deles sårbare erfaringer. Som illustreret på Figur 11, adskiller designet den relationelle dataintegritet fra den eksterne præsentation i brugerfladen - alle opslag er teknisk tilknyttet en valid bruger i databasen, mens de fremstår anonyme for fællesskabets øvrige medlemmer.

Inden oprettelsen delegeres til Service-laget, håndhæver Route Handleren et sikkerhedslag: brugere, der er globalt mutede eller mutede i det specifikke fællesskab, afvises med en 403-fejl, før forretningslogikken overhovedet aktiveres.

En central designbeslutning er synlighedshierarkiet i `PostService.getPostById()`, der evaluerer et `canSeeAnon`-flag baseret på kaldende brugers relation til forfatteren: forfatteren selv, globale administratorer, fællesskabsmoderatorer og brugerens venner kan alle se den reelle forfatteridentitet. For alle andre anvender systemet forfatterens unikke, persistente `CommunityAnonymousIdentity` - et autogenerated dyre-navn, emoji og baggrundsfarve - frem for en generisk "Anonym Bruger"-placeholder. Dette design er bevidst: den anonyme identitet er stabil inden for fællesskabet og giver læsere mulighed for at genkende og interagere med den samme anonyme forfatter på tværs af opslag, uden at den virkelige identitet afsløres.

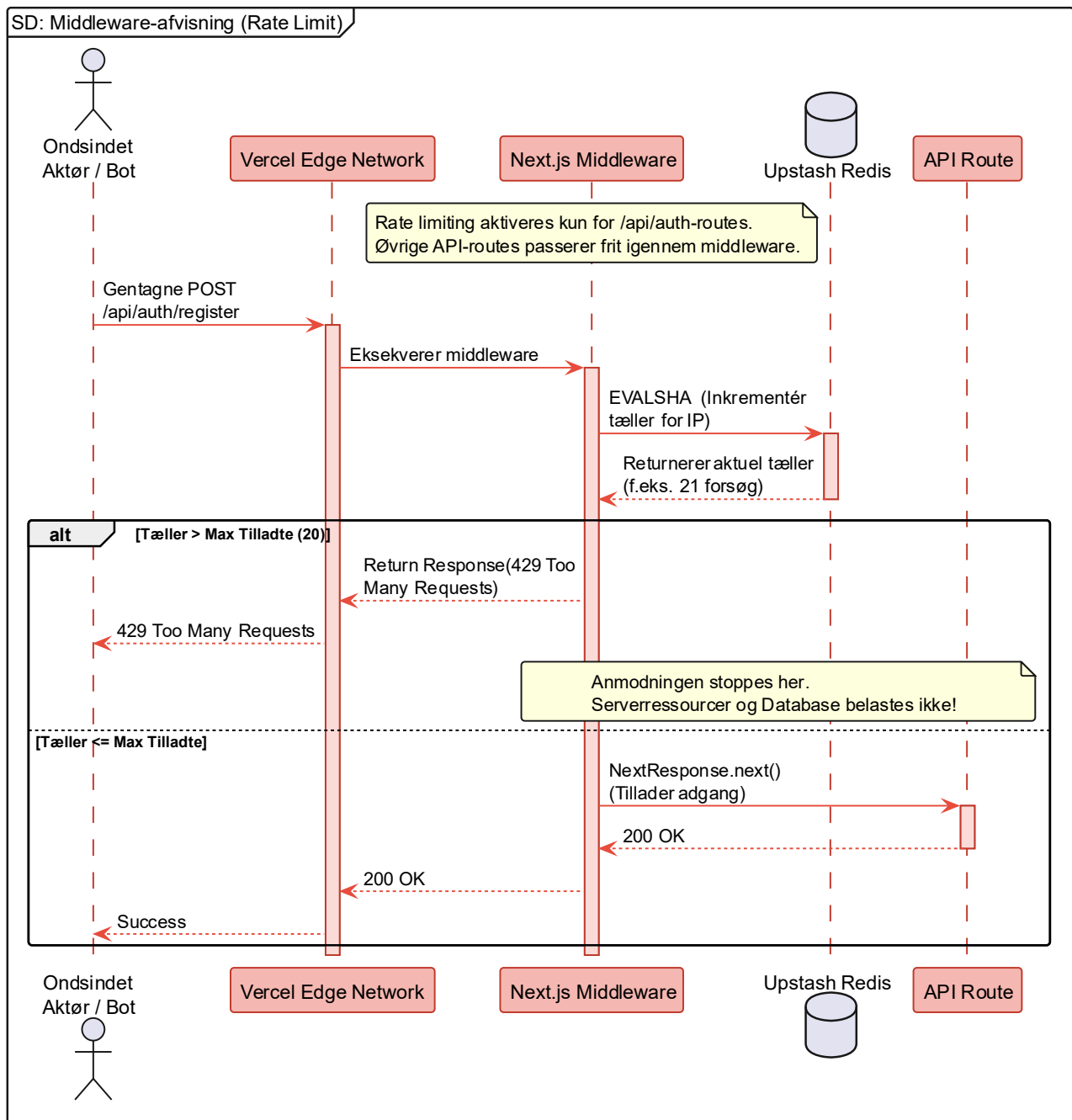


Figur 11: Sekvensdiagram for Opret Opslag Anonymt (US3.10). Illustrerer Route Handleren sikkerhedslag, der afviser mutede brugere inden service-kald, samt `PostService`'s synlighedshierarki, der afgør forfatterens identitet baseret på kaldende brugers relation (forfatter, admin, moderator, ven). For ikke-privilegerede læsere erstattes forfatterdata med brugerens persistente `CommunityAnonymousIdentity`, ikke en generisk placeholder.

5.5. SD: Middleware-afvisning (rate limit/CSRF) (NFR)

For at validere systemets robusthed (Supportability og Reliability) modelleres her i dette sekvensdiagram et sad path-scenarie, hvor en ondsindet aktør forsøger at overbelaste platformen (f.eks. via DDoS eller Brute Force mod login-endpointet). Som illustreret på Figur 12, validerer dette flow de forebyggende tiltag, der er opstillet i risikoanalysen (jf. R13: Sikkerhedshuller (3)).

Rate limiting er målrettet `/api/auth-routes`, der udgør det primære angrebsflade for credential-stuffing og brute-force-angreb. Diagrammet demonstrerer, hvordan Next.js Edge Middleware (11) i kombination med Upstash Redis (9, 10) fungerer som et fremskudt forsvarsværk, der håndhæver Fail Fast-princippet (31). Ved at evaluere anmodningen direkte på Edge-netværket - med IP-adressen som nøgle i Redis-tælleren - kan systemet afvise mistænkelig trafik, før den overhovedet belaster applikationens centrale processer eller PostgreSQL-databasen. Denne arkitektoniske isolering sikrer, at systemets kritiske ressourcer forbliver tilgængelige for legitime brugere, selv under eksternt pres.



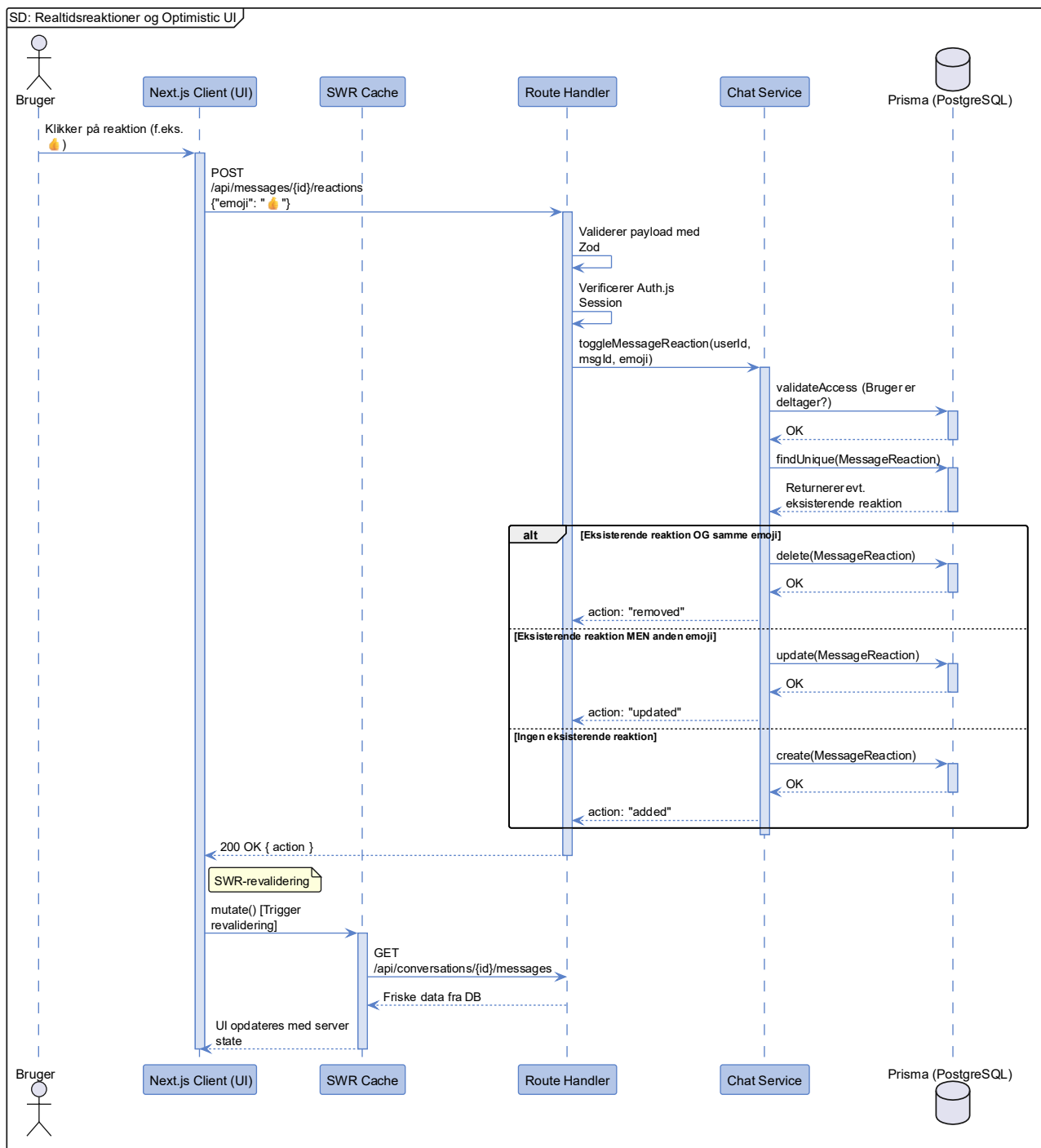
Figur 12: Sekvensdiagram for Edge Middleware-afvisning, Rate Limit (NFR). Modellerer håndteringen af et sad path-scenarie, hvor en ondsindet aktør gentagne gange rammer `/api/auth-routes` og blokeres ved grænsen til systemet via Upstash Redis-tæller pr. IP-adresse. Diagrammet belyser effektiviteten af Fail Fast-princippet, hvor belastningen minimeres gennem tidlig fejlhåndtering, inden applikations- og persistenslaget aktiveres.

Bemærk: Som en visuel detalje er diagrammets farvetema markeret med røde nuancer fremfor blå (Aarhus Universitets farvetema) for at understrege, at der her modelleres et fejlscenarie forårsaget af et ondsindet angreb.

5.6. SD: Realtidsreaktioner og Optimistic UI (US3.3)

Reaktionsflowet, som er illustreret på Figur 13, demonstrerer håndteringen af emoji-reaktioner i chatmodulet. Implementeringen følger et klassisk request/response-mønster via SWR's `mutate()`-funktion: klienten sender reaktionen til serveren, og UI'en opdateres først når serveren svarer og SWR har revalideret den lokale cache mod den faktiske databasetilstand.

Diagrammet illustrerer desuden, hvordan Service-laget håndterer reaktionernes tre mulige tilstande: tilføjelse, opdatering og fjernelse via en toggle-mekanisme. Dette sikrer, at `MessageReaction`-tabelens integritetsbetingelse om én aktiv reaktion pr. bruger pr. besked håndhæves konsistent.

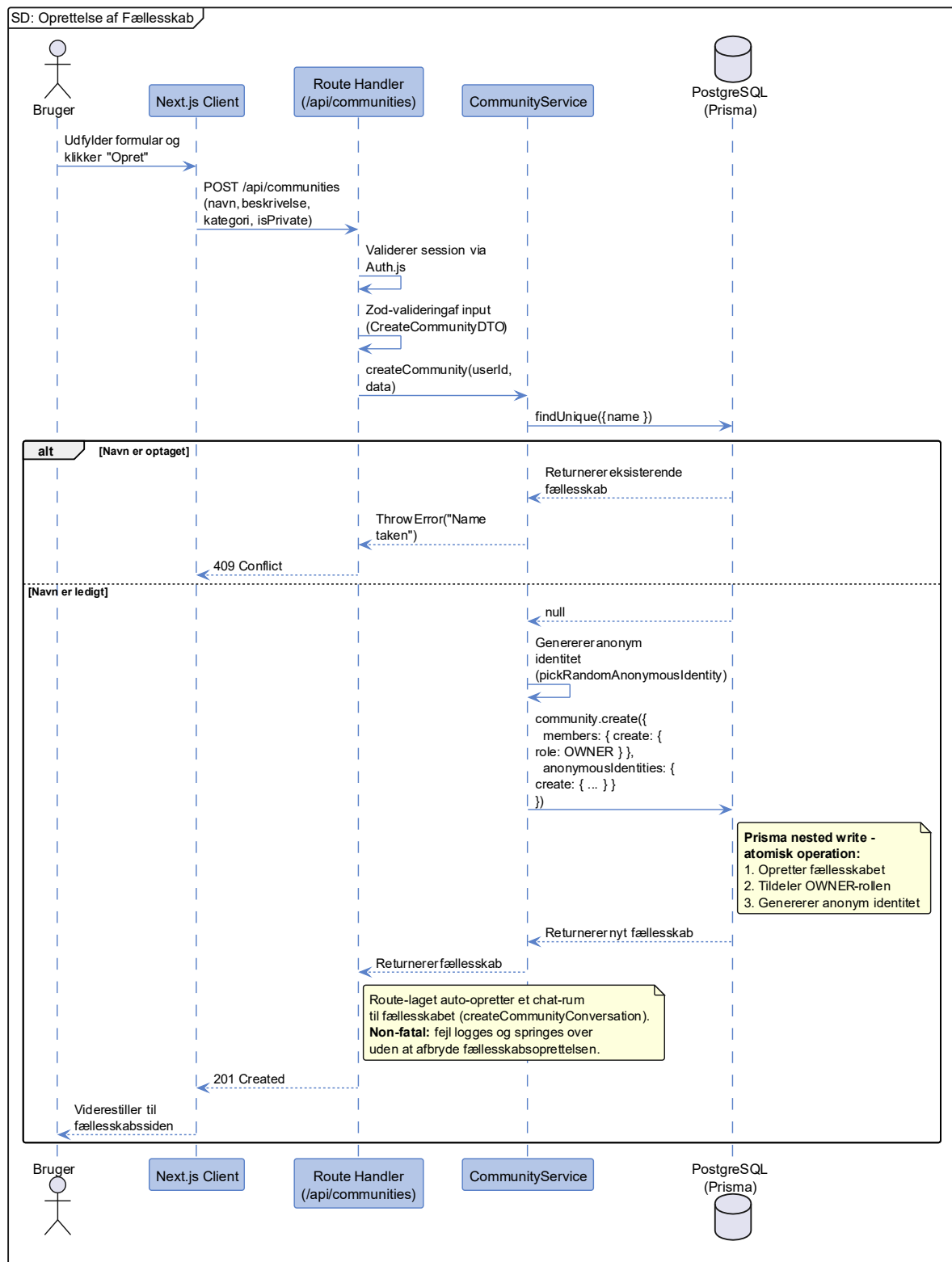


Figur 13: Sekvensdiagram for Toggle Besked-reaktion (US3.3). Illustrerer SWR's `mutate()`-revalidering efter server-bekræftelse samt Service-lagets toggle-mekanisme, der håndhæver integritetsbetingelsen om én aktiv reaktion pr. bruger pr. besked gennem tre mulige udfald: tilføjelse, opdatering og fjernelse.

5.7. SD: Oprettelse af fællesskab (US3.7)

Sekvensdiagrammet på Figur 14 illustrerer forløbet, når en bruger opretter et nyt fællesskab. En central designbeslutning fremgår her: Service-laget anvender en kombineret Prisma *nested write*, der atomisk udfører tre operationer i én og samme databaseforespørgsel: opretter fællesskabet, tildeler opretterens brugerkonto rollen som OWNER og genererer en anonym fællesskabsidentitet til ophavsmanden (16). Dette sikrer, at der aldrig kan eksistere et fællesskab uden en ophavsmand, og at skaberen øjeblikkeligt kan publicere anonymt i fællesskabet uden en separat opsætningsprocedure.

Som et efterfølgende, non-fatal trin auto-opretter Route Handleren desuden et dedikeret chat-rum til fællesskabet via `createCommunityConversation()`. Denne operation er bevidst isoleret fra den primære oprettelsestransaktion: hvis chat-oprettelsen fejler, logges fejlen, men fællesskabsoprettelsen afbrydes ikke. Brugeren viderestilles til fællesskabssiden uanset udfald.

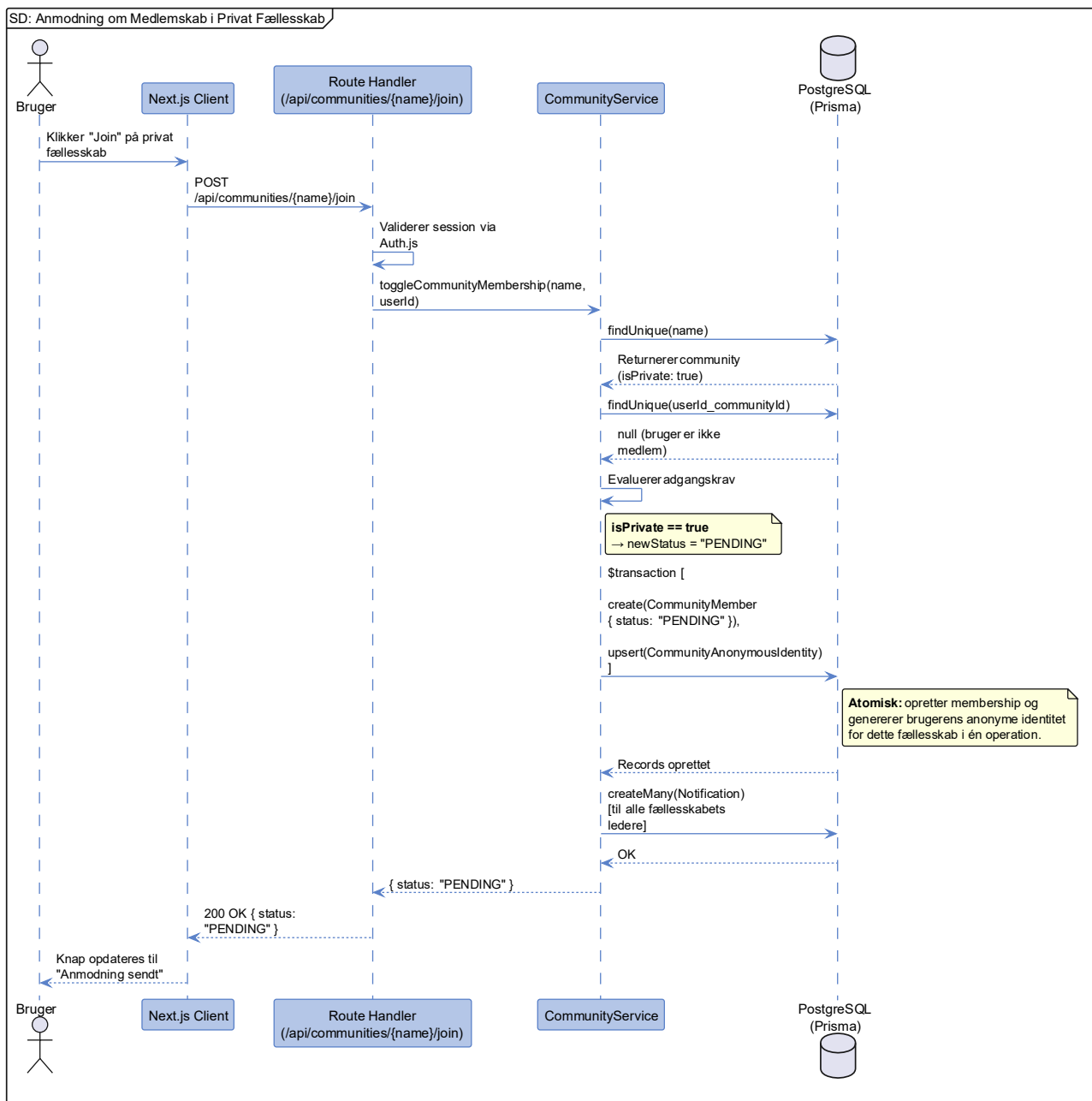


Figur 14: Sekvensdiagram for oprettelse af fællesskab (US3.7). Illustrerer Zod-valideringen i API-laget samt den atomiske Prisma nested write (16), der simultant opretter fællesskabet, tildeler OWNER-rolle til ophavsmanden og genererer dennes anonyme fællesskabsidentitet - alt i én samlet databaseoperation. Route Handleren auto-opretter desuden et fællesskabs-chatrum som et efterfølgende, non-fatal trin.

5.8. SD: Anmodning om medlemskab i privat fællesskab (US3.14)

Sekvensdiagrammet på Figur 15 viser den logiske forgrening i Service-laget, der håndterer join-anmodninger forskelligt afhængig af, om et fællesskab er konfigureret som privat. Denne skelnen er essentiel for at understøtte de to typologier af fællesskaber, som systemet understøtter: åbne fællesskaber (direkte accept) og private (afventer godkendelse).

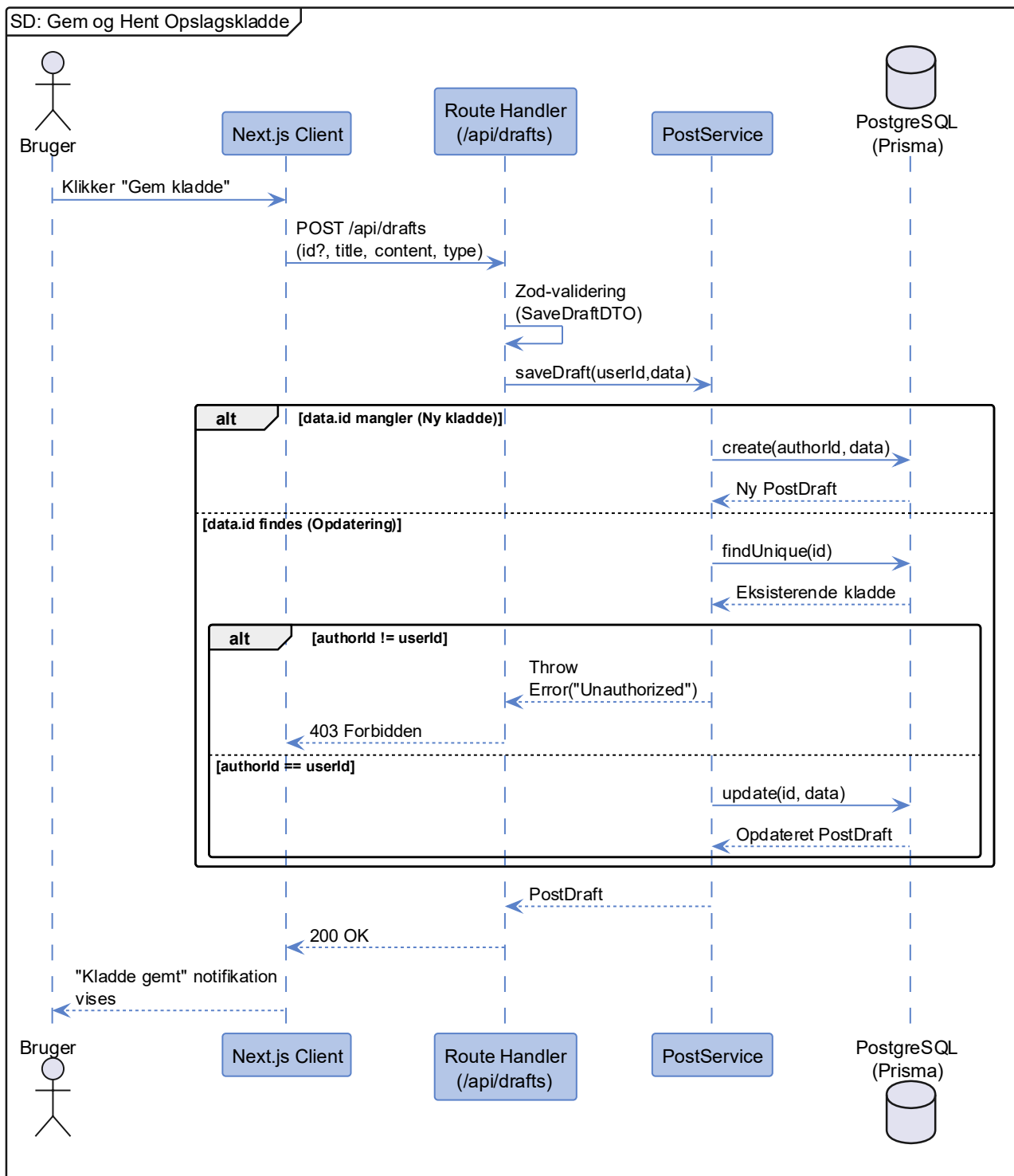
En designdetalje, der fremgår af diagrammet, er at join-operationen eksekveres som en atomisk `$transaction`: i én samlet databaseoperation oprettes både `CommunityMember`-recorden med den tildeelte status og brugerens `CommunityAnonymousIdentity` for det pågældende fællesskab. Den anonyme identitet er nødvendig for at brugeren øjeblikkeligt kan publicere anonymt, hvis de efterfølgende optages som aktivt medlem. Ydermere notificeres alle fællesskabets ledere (`OWNER`, `ADMIN`, `MODERATOR`) automatisk om den indkommende anmodning, så godkendelsesprocessen kan påbegyndes uden manuel overvågning.



Figur 15: Sekvensdiagram for anmodning om medlemskab i et privat fællesskab (US3.14). Belyser den dynamiske tilde-
ling af status (PENDING vs. ACCEPTED) baseret på fællesskabets isPrivate-konfiguration. Den atomiske \$transac-
tion opretter simultant CommunityMember-recorden og brugerens CommunityAnonymousIdentity, mens fællesska-
bets ledere notificeres automatisk om den indkomne anmodning.

5.9. SD: Gem og hent opslagskladde (afledt af US3.8)

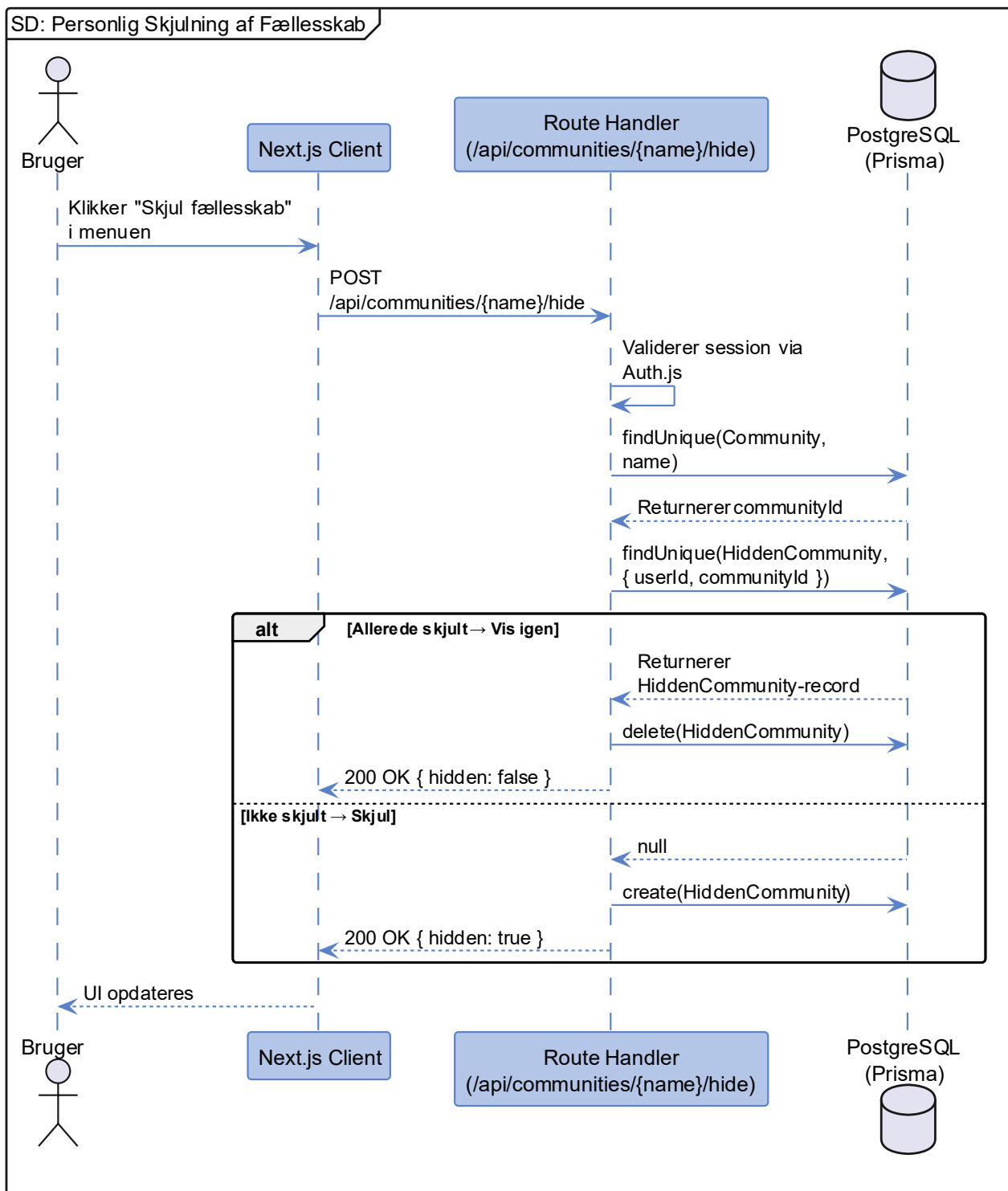
Sekvensdiagrammet på Figur 16 demonstrerer kladde-workflowet og fremhæver et væsentligt sikkerhedsprincip i Service-laget: ejerskabsvalidering. Inden en eksisterende kladde opdateres, verificeres det, at den autentificerede bruger, faktisk er forfatter til kladden. Dette forhindrer, at en bruger kan overskrive en anden brugers ufærdige indhold.



Figur 16: Sekvensdiagram for håndtering af opslagskladder. Fremhæver ejerskabsvalideringen i Service-laget, der sikrer, at kun kladdens forfatter kan opdatere indholdet.

5.10. SD: Personlig skjulning af fællesskab (afledt af US3.14/US2.13)

Sekvensdiagrammet på Figur 17 illustrerer toggle-mekanismen for personlig skjulning af fællesskaber. Som diagrammet viser, arbejder dette flow udelukkende på Route Handler-niveau uden delegation til et dedikeret Service-lag, da logikken er simpel nok til at leve direkte i handleren. Den toggle-baserede tilgang (skjul/vis) afspejles i det returnerede hidden-flag til klienten.



Figur 17: Sekvensdiagram for personlig skjulning af fællesskab (toggle). Demonstrerer *HiddenCommunity*-relationens rolle som en brugerspecifik præferencetabel, der opererer uafhængigt af fællesskabets globale *isActive*-tilstand.

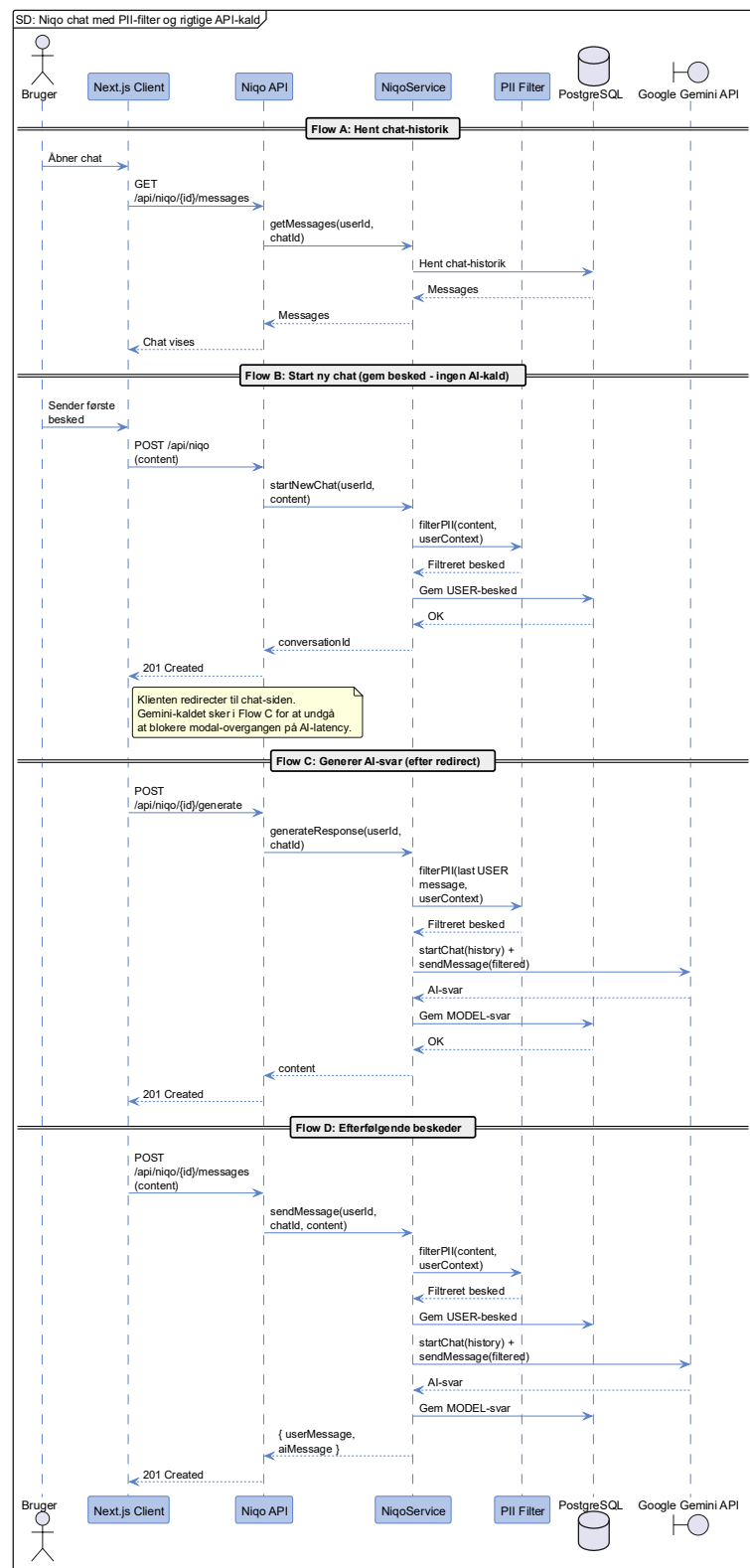
5.11. SD: AI-støttebot med privacy-filtrering (US3.19)

Sekvensdiagrammet på Figur 18 illustrerer interaktionen mellem brugeren, systemet og den eksterne Gemini LLM via Niqo-støttebotten. Flowet illustrerer Privacy by Design-princippet i praksis. Frem for at sende brugerens rå input direkte til en ekstern tredjepart, fungerer Service-laget og en dedikeret PII-filtreringsmekanisme (Regex) som et aktivt skjold.

Diagrammet belyser, hvordan personhenførbare oplysninger i brugerens indtastede tekst - herunder e-mailadresser, telefonnumre, CPR-numre og brugerens eget navn fra profilen - bortfiltreres og erstattes med sikre placeholders (f.eks. [NAME REMOVED]), før prompten videresendes til Google. Niqo har ingen adgang til platformens øvrige brugerdata og opererer udelukkende på baggrund af det, brugeren aktivt skriver.

En arkitektonisk designbeslutning fremgår af diagrammets opdeling i fire distinkte flows. Når en ny chat startes (Flow B, `POST /api/niqo`), gemmer systemet udelukkende brugerens besked og returnerer øjeblikkeligt et `conversationId` - uden at kalde Gemini API'et. Klienten omdirigerer til chat-siden og udløser derpå Flow C (`POST /api/niqo/{id}/generate`), der foretager det faktiske Gemini-kald og persisterer AI-svaret. Denne adskillelse er bevidst: den eliminerer latency-blokering af dialogvinduet overgang og giver brugeren en oplevelse af øjeblikkelig respons, mens AI-svaret hentes asynkront. Efterfølgende beskeder i den etablerede chat (Flow D) kombinerer brugerbesked, PII-filtrering og Gemini-kald i én sammenhængende serveroperation.

Samtidig illustreres det, hvordan systemet automatisk gemmer både brugerens og modellens beske-der i databasen for at opretholde en meningsfuld samtalekontekst, helt uden at gå på kompromis med datasikkerheden.



Figur 18: Sekvensdiagram for AI-støttebotten Niqo med PII Privacy-filtrering (US3.19). Diagrammet opdeler flowet i fire faser: hentning af historik (A), opstart af ny chat uden Gemini-kald (B), asynkron AI-svargenerering via Google Gemini API (C) og efterfølgende beskedhåndtering (D). PII-filtrering via Regex i Service-laget (28) sikrer at personhenførbare oplysninger fjernes fra alle prompts, inden de videresendes til den eksterne LLM.

6. API-design (grænseflader)

Dette afsnit specificerer den tekniske kontrakt mellem systemets klientlag og Next.js Route Handlers (API) (13). Da API'et fungerer som bindeleddet mellem frontenden og databasen, er det essentielt med en veldefineret struktur baseret på RESTful-principper (32), som sikrer en forudsigelig og skalerbar kommunikation mellem lagene.

6.1. REST API kontrakter

REST-kontrakterne, som er illustreret i Tabel 4, udgør et udtræk af de mest *Architecturally Significant* endpoints i systemet, opdelt i autentificering og kernedomæne. Som det fremgår af oversigten, kommunikerer API'et udelukkende via JSON-payloads og anvender standardiserede HTTP-statuskoder (33) (f.eks. 401 Unauthorized, 403 Forbidden og 429 Too Many Requests) for at sikre konsistent fejlhåndtering på klienten. Al sessionshåndtering varetages automatisk af Auth.js (7) via `HttpOnly`-cookies (8), hvorfor klienten ikke skal håndtere eksplicit autentificering i hver anmodning, da sessionen vedhæftes automatisk via browserens cookie-håndtering.

Tabel 4: Specifikation af centrale REST API-endpoints. Tabellen definerer den tekniske kontrakt for datakommunikationen mellem klient og server i Next.js-applikationen.

RESTful API-interface: Metoder, stier og sikkerhedsparametre				
Endpoint	Metode	Beskrivelse og sikkerhed	Request (Body / Params)	Forventet Response (Succes / Fejl)
/api/auth/register	POST	Opretter lokal bruger. Sikret via edgebaseret rate limiting.	Body: email, username, password, name	201: Created 400: Validation
/api/auth/complete-google	PUT	Onboarding af OAuth-brugere. Kræver Auth.js Session.	Body: username	200: OK 401: Unauthorized 409: Conflict
/api/daily-status	POST	Registrerer/Opdaterer dagsform. Kræver Auth.js Session. Zod-validering af input.	Body: value (Int), note (String)	201: DailyStatus objekt 401: Unauthorized
/api/daily-status	GET	Henter historik for den autentificerede bruger.	Query: ?days=30	200: Liste af DailyStatus 401: Unauthorized
/api/communities/{name}/posts	POST	Opretter opslag. isAnonymous styrer forfatter-maskering.	Param: id Body: content, isAnonymous	201: Post objekt 401: Unauthorized
/api/communities/{name}/posts	GET	Henter feed. Backendten filtrerer author-data automatisk, hvis opslaget er anonymt.	Param: id Query: ?page=1	200: Array af Post DTO'er 404: Not Found

/api/communities	GET	Henter brugerens fællesskabsoverblik (alle, anbefalede, tilmeldte, skjulte). Kræver Auth.js Session.	Ingen	200: Objekt med filtrerede lister 500: Internal Error
/api/communities	POST	Opretter nyt fællesskab og tildeler automatisk creator-rollen som OWNER. Zod-validering.	Body: name, description, category, isPrivate	201: Community 400: Validation failed 409: Name taken
/api/communities/{name}	GET	Henter detaljeret data for et specifikt fællesskab inkl. medlemskabsstatus og isPersonallyHidden.	Param: name	200: Community detaljer 403: Inaktivt fællesskab 404: Not found
/api/communities/{name}/join	POST	Toggler medlemskab. Sætter status til PENDING for private og ACCEPTED for offentlige fællesskaber.	Param: name	200: { status } 404: Not found
/api/communities/{name}/hide	POST	Toggler personlig skjulning af fællesskab for den autentificerede bruger.	Param: name	200: { hidden: bool } 404: Not found
/api/posts	POST	Opretter et nyt opslag. Validerer fællesskabsmedlemskab. isAnonymous maskerer forfatter i feed.	Body: title, type, content, isAnonymous, communityId?	201: Post oprettet 400: Validation failed 403: Ikke godkendt medlem

/api/drafts	GET	Henter alle brugerens kladder sorteret efter senest opdateret.	Ingen	200: Liste af PostDraft 401: Unauthorized
/api/drafts	POST	Opretter ny kladde (uden id) eller opdaterer eksisterende kladde (med id). Ejerskab valideres i service.	Body: id?, title?, content, type, communityId?	200: Draft 400: Validation failed 403: Unauthorized
/api/drafts/{id}	DELETE	Sletter kladde permanent. Ejerskab verificeres af Service-laget inden sletning.	Param: id	200: Success 403: Unauthorized 404: Not found

Bemærk: Implementeringen udnytter Next.js Route Handlers, hvilket eliminerer behovet for en separat backend-instans. Autentificering og sessionsstyring forvaltes af Auth.js, som sikrer, at følsomme data (cookies) håndteres som HttpOnly og SameSite=Lax.

7. Referencer

1. Gregersen T. Vejledning til udfærdigelse af projektrapporter - version 1.5. Vejledningsdokument. Ingeniørhøjskolen Aarhus Universitet; 2018.
5. Brown S. The C4 model for visualising software architecture [Internet]. [år ukendt]. [hentet 27. februar 2026]. Tilgængelig fra: <https://c4model.com/>.
6. Vercel. Next.js Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://nextjs.org/docs>.
7. Auth.js contributors. Auth.js: Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://authjs.dev/getting-started>.
8. The OWASP Foundation. HttpOnly [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://owasp.org/www-community/HttpOnly>.
9. Redis Inc. Redis - The open source, in-memory data store [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://redis.io/>.
10. Upstash Inc. Upstash - Serverless Data Platform [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://upstash.com/>.
11. Vercel. Next.js: Edge Runtime [Internet]. [år ukendt]. [hentet 1. marts 2026]. Tilgængelig fra: <https://nextjs.org/docs/app/api-reference/edge>.
12. Richards M. Software Architecture Patterns - Layered Architecture [Internet]. 2015. [hentet 2. marts 2026]. Tilgængelig fra: <https://www.oreilly.com/content/software-architecture-patterns/>.
13. Vercel. Next.js: Route Handlers [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://nextjs.org/docs/app/getting-started/route-handlers>.
14. Fowler M. Data Transfer Object [Internet]. 2004. [hentet 8. marts 2026]. Tilgængelig fra: <https://martinfowler.com/eaCatalog/dataTransferObject.html>.
15. McDonnell C. Zod: TypeScript-first schema validation with static type inference [Internet]. [år ukendt]. [hentet 8. marts 2026]. Tilgængelig fra: <https://zod.dev/>.
16. Prisma Data. Prisma Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.prisma.io/docs>.
17. Vercel. Vercel Documentation [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://vercel.com/docs>.
18. Neon. Neon Documentation [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://neon.com/docs/introduction>.

19. PostgreSQL Global Development Group. PostgreSQL 18.2 Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.postgresql.org/docs/18.2/index.html>.
20. Auth.js contributors. Auth.js: Prisma Adapter [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://authjs.dev/getting-started/adapters/prisma>.
21. Prisma Data. Relations - Prisma ORM Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.prisma.io/docs/orm/prisma-schema/data-model/relations>.
23. Elliott E. CUID: Collision-resistant ids optimized for horizontal scaling and performance [Internet]. [år ukendt]. [hentet 28. februar 2026]. Tilgængelig fra: <https://github.com/paralleldrive/cuid>.
24. The OWASP Foundation. Insecure Direct Object Reference Prevention Cheat Sheet - OWASP Cheat Sheet Series [Internet]. [år ukendt]. [hentet 3. marts 2026]. Tilgængelig fra: https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html.
25. Europa-Parlamentet og Rådet for Den Europæiske Union. Europa-Parlamentets og Rådets forordning (EU) 2016/679 af 27. april 2016 om beskyttelse af fysiske personer i forbindelse med behandling af personoplysninger og om fri udveksling af sådanne oplysninger (databeskyttelsesforordningen) [Internet]. 2016. [hentet 3. marts 2026]. Tilgængelig fra: <https://eur-lex.europa.eu/legal-content/DA/TXT/?uri=CELEX%3A32016R0679>.
26. Sakimura N, Bradley J, Agarwal N, Jay E. RFC 7636: Proof Key for Code Exchange by OAuth Public Clients [Internet]. 2015. [hentet 22. februar 2026]. Tilgængelig fra: <https://datatracker.ietf.org/doc/html/rfc7636>.
27. Vercel. Next.js: Server Actions [Internet]. [år ukendt]. [hentet 27. februar 2026]. Tilgængelig fra: <https://nextjs.org/docs/app/getting-started/updating-data>.
28. Google LLC. Gemini API - Google AI for Developers [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://ai.google.dev/gemini-api/docs>.
29. Wirtz D. bcryptjs - npm [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.npmjs.com/package/bcryptjs>.
30. Prisma Data. Prisma: CRUD [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.prisma.io/docs/orm/prisma-client/queries/crud>.
31. Fowler M. Fail Fast [Internet]. 2004. [hentet 28. februar 2026]. Tilgængelig fra: <https://martinfowler.com/ieeeSoftware/failFast.pdf>.
32. IBM. What is a REST API? [Internet]. 24. april 2025. [hentet 9. marts 2026]. Tilgængelig fra: <https://www.ibm.com/topics/rest-apis>.
33. MDN Web Docs. HTTP response status codes [Internet]. [år ukendt]. [hentet 9. marts 2026]. Tilgængelig fra: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>.

8. Bilag

2. Villadsen MD. Chroniqo: Teknologianalyse. Bilag 4. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
3. Villadsen MD. Chroniqo: Risikoanalyse. Bilag 3. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
4. Villadsen MD. Chroniqo: Domæneanalyse. Bilag 2. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
22. Villadsen MD. Chroniqo: Kravspecifikation. Bilag 1. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.